UltraKMeans: Fully Scalable and High Quality k-means

Xiaojun Dong*†
University of California, Riverside
Riverside,, USA
xdong038@ucr.edu

Kishen N Gowda*
University of Maryland
College Park,, USA
kishen19@cs.umd.edu

Jessica Shi†
Google Research
New York,, USA
jessicashi@gmail.com

MohammadHossein Bateni
Google Research
New York,, USA
bateni@google.com

Laxman Dhulipala
University of Maryland
College Park,, USA
laxman@umd.edu

Xiaoen Ju
Google Research
New York,, USA
xiaoen@google.com

Jakub Łącki
Google Research
New York,, USA
jlacki@google.com

Abstract

As k-means clustering is applied to massive datasets, state-of-the-art implementations of k-means must scale simultaneously along four axes: the input size n, the dimensionality d, the number of clusters k, and the thread count. However, achieving this scalability while ensuring high clustering quality can be hindered by the seeding phase. The canonical k-means++ seeding is inherently sequential in k, and although its parallel variant k-means‖ removes this dependency, it reintroduces a sequential $\Theta(k^2)$ downsampling step that becomes a significant bottleneck for large k.

To overcome these challenges, we present UltraKMeans, an end-to-end k-means system that scales along all four axes. We design Doubling k-means, a novel parallel seeding algorithm that geometrically grows the D^2-sampling batch across $O(\log k)$ rounds, ensuring excellent parallelization while closely matching the clustering quality of k-means++. We also give the first scalable implementation of Hierarchical Agglomerative Clustering with Ward's linkage (Ward's HAC) as a seeder, and show that its seeding alone almost always surpasses the quality of other seeding methods at a moderate cost. We provide an in-depth evaluation of UltraKMeans on 12 real-world datasets of varying dimensionality, spanning 70 K to 2.77 B points. At k=10,000, Doubling k-means is 2.9–17× faster end-to-end than k-means++ and k-means‖ across our datasets, while matching their final SSE to within a few percent on average. Its seeding phase is 6.3–45× faster than those of k-means++ and k-means‖. Ward's HAC achieves the lowest SSE on 22 of 24 instances across two representative cluster counts (k=1000 and k=10,000), at the cost of increased runtime.

*Both authors contributed equally to this research.
†Work done in part while interning at Google Research.

1 Introduction

k-means is one of the most widely used clustering primitives in data analysis and unsupervised learning [48, 52]. Its workloads span the full range of dimensionalities. At low dimensions, k-means clusters GPS trajectories and points of interest in geospatial systems [34, 70], groups particles and halos in cosmological simulations [44, 59], and summarizes readings from sensor and smart-grid networks. At high dimensions, it organizes the learned embeddings used in modern retrieval, recommendation, and feature-learning pipelines for image, text, and multimodal data [19, 65]. The two regimes also overlap inside the same systems: for instance, approximate nearest-neighbor indexes embed k-means in both low- and high-dimensional regimes: product quantization partitions each embedding into low-dimensional subspaces and learns a small k-means codebook within each [40], while an inverted-file (IVF) index clusters high-dimensional embeddings with a *very large* k to partition points into Voronoi cells [25]. Across this spectrum, modern applications share three common characteristics: (1) *clustering many points*, (2) *clustering into a large number of clusters*, and (3) *execution on multi-core servers*, demanding implementations that scale along all four axes (n, d, k, and thread count).

Formally, given n points $P = \{p_1, \ldots, p_n\} \subset \mathbb{R}^d$ and a target number of clusters k, k-means seeks k centroids that minimize the *Sum of Squared Errors (SSE)*, the total squared distance from each point to its nearest centroid. Minimizing SSE is NP-hard [5], so practical solvers follow a two-phase pipeline. A *seeding* phase selects k initial centroids, and a *refinement* phase, typically Lloyd's algorithm [48], iteratively reassigns points to their nearest centroid and recomputes centroids until convergence. Both phases are of critical importance to the quality of the final solution, since refinement only ever reaches a *local* minimum, and which minimum it reaches is largely determined by the seeding.

To organize the design space and to position prior work, it is useful to view k-means along two axes: the two *phases* above (seeding and refinement), and four *dimensions of scalability*: the input size n, the dimensionality d, the number of clusters k, and the thread count p. Existing methods typically optimize a single phase, or scale along only a subset of the four dimensions. However, the real-world workloads discussed above stress *all of them* at once, and, as we discuss next, no prior approach scales along all four dimensions while preserving clustering quality.

A long line of work has focused on accelerating Lloyd's refinement algorithm [21, 26, 35, 41, 54, 55]. Each Lloyd's iteration compares all n points against all k centers, costing $O(nk)$ work[1] and becoming the bottleneck as k grows. Pruning methods based on the triangle inequality, such as Elkan's [26] and Hamerly's [35], cut distance computations for small k, but Elkan's per-point bookkeeping itself scales with k, and both methods' pruning weakens as centers crowd together. More recently, Marigold [55] reformulates Lloyd's around cache-friendly, SIMD-optimized exhaustive search, scaling well with the dimension d but remaining linear in k. Crucially, no amount of refinement speedup helps the seeding phase—and on datasets where the SSE landscape has many well-separated local minima, even unlimited refinement cannot escape the poor basin that a low-quality seeding can fall into.

Yet high-quality seeding is itself hard to scale: most high-quality seeders parallelize poorly as k grows large. The canonical seeder, k-means++ [7], selects centers one at a time via D^2-sampling (each new center is drawn with probability proportional to its squared distance to the nearest already chosen center), achieving an $O(\log k)$ approximation and strong empirical quality. However, its k rounds are *strictly sequential*: the i-th center depends on the $(i-1)$-th, so the parallel span grows linearly in k. k-means‖ [12] parallelizes seeding by oversampling $\Theta(k)$ candidates per round over a few rounds and then downsampling back to k by running a weighted k-means++ on the candidate set. This removes the per-center dependency, but the downsampling reintroduces a sequential, $\Theta(k^2)$ work reclustering step that becomes the new bottleneck exactly when k is large. In short, both of these high-quality seeding algorithms suffer from the ***k-scaling bottleneck*** which prevents them from scaling along the k axis while preserving k-means++-quality seeds.

Our approach. To close this gap, we present UltraKMeans, an end-to-end k-means system that provides highly optimized implementations of both established methods and our new seeders, achieving state-of-the-art speed, quality, and scalability along all four dimensions without sacrificing clustering quality.

[1]We suppress d throughout the paper as distance computations can be trivially parallelized with $O(d)$ work and $O(\log d)$ span.

Our first contribution is a novel parallel seeder, Doubling k-means, that scales to large cluster counts. It scales gracefully with the cluster count k by applying the "prefix-doubling" idea from parallel algorithm design [17, 63, 64] to D^2-sampling. Rather than drawing one center at a time (k-means++) or oversampling and then requiring a sequential $\Theta(k^2)$ downsampling step (k-means‖), Doubling k-means grows the D^2-sampling batch geometrically over $O(\log k)$ rounds: it draws small batches of candidates in early rounds, when the distance distribution is still volatile, and larger batches in later rounds, once it has stabilized. This eliminates the downsampling step entirely, removing k-means‖'s $\Theta(k^2)$ bottleneck while empirically matching the quality of the k-means++ family.

Our second contribution is the first scalable implementation of Ward's HAC. Hierarchical agglomerative clustering with Ward's linkage (Ward's HAC) is a classic clustering method. Existing experiments on small scale data indicate that it delivers good quality seeds [37]. However, it has seen little practical use at scale since standard exact implementations of Ward's HAC require $\Theta(n^2)$ time, and the approximate subquadratic algorithm [2] has, to our knowledge, lacked a practical realization. In this paper, we provide a very fast and practical implementation of Ward's HAC seeding, together with the first large-scale empirical study of its performance as a k-means seeder. Concretely, we build on the approximate-Ward's HAC framework of Abboud et al. [2]. We make this framework practical for large datasets by running it on a subsample of the data rather than all n points, and by replacing their theoretical LSH structures with dynamically maintained, weight-partitioned Vamana graphs [62] to generate candidate merges.

Empirically, our new implementation of Ward's HAC produces seeds of such high quality that they are often *close to the best known solutions before any refinement at all*: it attains the lowest final SSE on 22 of 24 dataset–k instances at the cost of increased runtime, and its seeds alone come within 4.5% of the best post-refinement SSE on average. This makes Ward's HAC a quality-oriented complement to Doubling k-means: it is more expensive, but it exposes how much initialization quality is achievable when the seeding phase is allowed to perform more work.

Our third contribution is UltraKMeans, a highly parallel and efficient k-means system that achieves state-of-the-art speed, quality, and scalability along all four dimensions, supporting a full suite of seeding and refinement methods. Our system is carefully optimized and parallelized, so practitioners can choose the seeder–refiner combination that fits their workload without sacrificing performance. It offers a suite of seeders (Doubling k-means, Ward's HAC, k-means++, k-means‖, and uniform-random), paired with an approximate nearest-neighbor (ANNS) Lloyd's refinement engine that warm-starts each query from its previous-iteration cluster assignment to cut search cost. We

perform an extensive experimental evaluation of UltraKMeans on twelve real-world datasets spanning 70 K to 2.77 B points. At k=10^4, UltraKMeans with Doubling k-means is 2.9–17× faster end-to-end than its k-means++ and k-means|| configurations, while matching their final SSE to within a few percent. Across the tested dimensional regimes and cluster counts, it is also 3.8–9.0× faster than faiss-Random. faiss-AFK-MC^2 improves initialization quality but incurs additional sequential cost. Meanwhile, Ward's HAC attains the lowest final SSE on 22 of 24 instances, at the cost of increased runtime. UltraKMeans scales gracefully on full inputs to 100M points and obtains high-quality clusterings on datasets as large as 2.77B points in its default sampled-training mode. We release our code anonymously [8].

2 Preliminaries

Computational Model. We use the standard fork-join model with binary forking [16, 20]. We assume a set of threads that share a common memory. A process can `fork` two child software threads to work in parallel. When both children complete, the parent process continues. The *work* W of a computation is the total number of operations it performs, and the *span* D is the length of its longest chain of sequential dependencies. We can execute the computation using a randomized work-stealing scheduler in practice in $W/p + O(D)$ time with high probability on p processors [6, 18, 32]. Our algorithms are lock-free and internally deterministic for fixed random seeds [15], and use the standard parallel Reduce, Scan (which computes prefix sums), and Filter primitives, each of which takes $O(n)$ work and polylogarithmic span on an n-element array and is provided by libraries such as ParlayLib [14].

Problem Definitions. Given an input of n points $P \subseteq \mathbb{R}^d$ and the number of desired centers k, the goal of a k-means algorithm is to return k points A to minimize the cost given by:

$$\text{cost}(P, A) = \Sigma_{p \in P} \min_{c \in A} ||p - c||^2$$

k-means is known to be an NP-hard problem [5]. While theoretical works such as local search variants [4, 42] offer constant-factor approximation guarantees, they typically require expensive local-search procedures with high computational cost. In this paper, we focus on scalability and system efficiency; thus, we compare against standard heuristics like Lloyd's algorithm and k-means++, which are the de facto standards for large-scale clustering.

2.1 The Standard k-means Framework

k-means algorithms typically follow the two-phase pipeline of Alg. 1 [7, 12, 55]. A *seeding* phase, `InitCentroids` (Line 1), selects k initial centers and returns their initial point assignments. Starting from this assignment, each refinement iteration calls `UpdateCentroids` (Line 3) to set every center to the mean of its current cluster, followed by `AssignClusters` (Line 4) to reassign points to the updated centers. The assignment step is often the computational bottleneck. In our system, `InitCentroids` can be instantiated with standard baselines (e.g., k-means++, k-means||, or uniform random seeding) as well as our scalable Doubling k-means (Sec. 3) and quality-oriented Ward's HAC (Sec. 4) seeders, and the `AssignClusters` bottleneck is accelerated using an efficient ANNS data structure, as described in Sec. 4.2.

Algorithm 1: Unified Parallel k-means Framework

Input: Dataset $P \in \mathbb{R}^{n \times d}$, Number of clusters k, Number of iterations T
Output: Centroids C, Assignments α

// Seeding Phase
1 $C^{(0)}, \alpha^{(0)} \leftarrow$ `InitCentroids`(P, k)
2 **for** $t \leftarrow 1$ **to** T **do**
  // Step 1: Recompute centers from the current assignment
3 $C^{(t)} \leftarrow$ `UpdateCentroids`$(P, \alpha^{(t-1)})$
  // Step 2: Reassign points using the preceding assignment as optional search state
4 $\alpha^{(t)} \leftarrow$ `AssignClusters`$(P, C^{(t)}, \alpha^{(t-1)})$
5 **return** $C^{(T)}, \alpha^{(T)}$

2.2 Existing Seeding Algorithms

k-means++. The k-means++ [7] algorithm is the canonical sequential instantiation of `InitCentroids`. It selects centers iteratively in k strictly dependent rounds. In the first round, it selects a random centroid out of the n input points and initializes the squared distance (D^2) from each point to this center. In each subsequent round, a new center is sampled with probability proportional to the minimum squared distance (D^2) to any currently selected center. To parallelize this approach, one updates the D^2 distances for each point (a parallel map) and constructs the sampling distribution using a parallel Scan primitive. While these inner operations can be efficiently parallelized, the outer loop (k iterations) is inherently sequential. Therefore, for modern large-cluster workloads (e.g., $k \geq 10^4$), this linear dependency on k serializes the execution flow.

k-means||. To mitigate the sequential bottleneck of k-means++, k-means|| [12] proposes a parallel oversampling instantiation of `InitCentroids`. Instead of selecting one center per round, it samples an expected number of ρk candidates (where $\rho > 1$ is an oversampling factor) in parallel over a small constant number of rounds. This successfully reduces the span of candidate sampling. However, the framework requires returning exactly k centers. To satisfy this, k-means|| employs a final reduction step to downsample the selected candidates, which is typically implemented by running a weighted k-means++ on the set of oversampled candidates. Crucially, the use of k-means++ reintroduces the sequential dependency discussed above. For large k, this bottleneck is prohibitive: sequentially reclustering $O(\rho k)$ candidates down to k final centers incurs $\Theta(k^2)$ work, effectively shifting the scalability bottleneck

from the dataset size n to the cluster count k. We use this fixed-round formulation throughout the main paper.

HAC-based initializers. Ward's HAC can be seen as a greedy algorithm for minimizing the k-means objective: starting from singleton clusters, one repeatedly merges the pair of clusters chosen such that the increase of the k-means objective is minimized. The algorithm progresses until exactly k clusters remain.

Even though the algorithm has been observed to produce good quality results [37], existing implementations suffer from poor scalability. Standard exact implementations of Ward's HAC maintain pairwise cluster distances in quadratic time and memory, and any exact algorithm requires $\Omega(n^2)$ time under standard fine-grained complexity assumptions [2]. Meanwhile, the subquadratic approximate framework of Abboud et al. [2] relies on dynamic nearest-neighbor search and has not been evaluated as a practical large-scale k-means initializer. This motivates our implementation study of Ward's HAC in Sec. 4.

2.3 Existing Refinement Algorithms

Lloyd's Algorithm. Lloyd's algorithm [48] is the standard instantiation of the refinement loop. Each iteration performs two steps. Given the current point assignments, `UpdateCentroids` first sets each center to the arithmetic mean of its assigned points using $O(n)$ work and $O(\log n)$ span. `AssignClusters` then computes the nearest updated center for every point using $O(nk)$ work and $O(\log n)$ span. Both steps parallelize naturally over points. The algorithm converges when no assignments change, though in practice a fixed number of iterations (typically 10–25) is used. Because every iteration examines all $n \times k$ point-center pairs, Lloyd's becomes a bottleneck when k is large.

Accelerated Variants. Elkan's algorithm [26] and Hamerly's algorithm [35] reduce the number of distance computations per iteration by maintaining upper and lower bounds on point-center distances and using the triangle inequality to skip redundant comparisons. Elkan's algorithm maintains a lower bound for every point-center pair, which requires $\Theta(nk)$ auxiliary memory in addition to center-center distance information. At the large n and k targeted in this paper, storing and updating these bounds requires a substantial memory footprint and incurs considerable memory traffic. Hamerly's algorithm reduces the per-point state to a constant number of bounds, but its coarser bounds prune fewer distance computations as k grows and as distances concentrate in high dimensions. Both methods remain parallelizable over points. Their limitation in our setting is instead that bound maintenance and irregular memory access can outweigh the distance computations they eliminate. Yinyang k-means [24] also uses group-level lower bounds to prune point-center comparisons while preserving exact Lloyd assignments. We therefore focus on Lloyd's as the baseline refinement algorithm and introduce alternative acceleration strategies in Sec. 4.2. Concurrent work also targets high-dimensional, large-k refinement: SuperKMeans [43] prunes dimensions after partial-distance GEMM rather than searching a center graph, while SHEESH [61] uses preceding Lloyd assignments to seed graph search.

Seeder	k-means++	k-means‖	Doubling k-means
Rounds	k	$O(1)$	$O(\log k)$
Sequential bottleneck	k sequential picks	$\Theta(k^2)$ downsampling	none
Work	$O(nk)$	$O(nk)$	$O(nk)$
Span	$O(k \log n)$	$O(k \log k + \log n)$	$O(\log n \log k)$

Table 1. D^2-sampling seeders along the scalability axes.

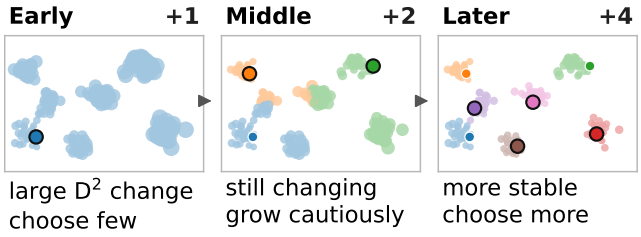


Figure 1. Doubling k-means illustration. It uses small early batches while the D^2 distribution can change substantially, then grows the batch size once the distribution is more stable. Dark outlines mark centers selected in the current round.

2.4 Organization of the Paper

In the following sections, we present seeding algorithms that achieve high parallelism without the k^2 reclustering cost (Sec. 3) or deliver exceptionally high quality seeds (Sec. 4), and explore the use of parallel approximate nearest neighbor search (ANNS) to accelerate the per-iteration assignment step of Lloyd's (Sec. 4.2). We present additional related work in appendix A.

3 Doubling k-means Seeding

In this section, we present Doubling k-means (Alg. 2), an efficient parallel seeding algorithm that scales to large cluster counts without the $\Theta(k^2)$ downsampling cost of k-means‖. Our algorithm builds on the prefix-doubling idea widely used in parallel algorithm design [17, 63, 64], based on a simple intuition: the clustering induced by the chosen centers changes substantially in the early rounds, when each new center can reshape large regions, but becomes increasingly stable as more centers accumulate (Fig. 1). Doubling k-means exploits this stability with a *geometric batch schedule*: it samples a small initial batch of centers via D^2-sampling in the first round and multiplies the batch size by α each round, so the cumulative number of centers reaches k in $O(\log k)$ rounds. The small early batches refresh the D^2 distribution

Algorithm 2: Doubling k-means Seeding Algorithm

Input: Point set $P = \{p_1, \ldots, p_n\}$, number of clusters k, growth factor $\alpha > 1$
Output: Initial centers C, assignments σ

// Round 0: pick one random center and initialize distances
1 let $c_0 = p_j$ where j is chosen uniformly at random from $[n]$
2 $\delta[i] \leftarrow d(p_i, c_0)^2$ and $\sigma[i] \leftarrow c_0$ for all $i \in [n]$ in parallel
3 $C \leftarrow \{c_0\}$; $\Phi \leftarrow \textsc{Reduce}(\delta, +)$; $b \leftarrow 1$ *// b is the batch target*
4 **while** $|C| < k$ **do**
5 $b \leftarrow b \cdot \alpha$; $b_r \leftarrow \min(\lfloor b \rfloor, k - |C|)$
// Parallel D^2-sampling with replacement
6 $S \leftarrow D^2\texttt{-Sample}(P, \delta, \Phi, b_r)$
7 $C \leftarrow C \parallel S$ *// append sampled centers*
8 $\Phi \leftarrow \texttt{UpdateDistances}(P, S, \delta, \sigma)$
9 **return** C, σ
10 **Function** $D^2\texttt{-Sample}(P, \delta, \Phi, b)$:
11 **foreach** $t \in [b]$ **do** *// in parallel*
12 draw $j_t \in [n]$ with probability $\delta[j_t]/\Phi$
13 **return** $\langle p_{j_t} : t \in [b] \rangle$
14 **Function** $\texttt{UpdateDistances}(P, S, \delta, \sigma)$:
15 **foreach** $i \in [n]$ **do** *// in parallel*
16 **foreach** $c \in S$ **do**
17 **if** $d(p_i, c)^2 < \delta[i]$ **then**
18 $\delta[i] \leftarrow d(p_i, c)^2$; $\sigma[i] \leftarrow c$
19 **return** $\textsc{Reduce}(\delta, +)$ *// new total cost Φ*

when each new center is most likely to reshape it, while the geometrically larger later batches keep the number of rounds logarithmic once the distribution has become more stable. Tab. 1 contrasts Doubling k-means with k-means++ and k-means‖.

3.1 Algorithm

Alg. 2 gives the pseudocode for Doubling k-means. It takes a point set P of n points, the number of clusters k, and a growth factor $\alpha > 1$, and maintains a distance array δ and an assignment array σ, where $\delta[i]$ is the squared distance from p_i to its nearest selected center and $\sigma[i]$ is that center's index. After picking one uniformly random center and initializing δ and σ against it (Line 2), the algorithm proceeds in rounds until k centers are selected. In round r, it chooses a geometrically larger batch target, capped by the number of centers still needed, and draws that batch by parallel D^2-sampling (Line 6): using the current cost $\Phi = \sum_i \delta[i]$, it draws b_r independent centers from the fixed distribution with probabilities $\delta[i]/\Phi$, allowing repeated points. It appends the sampled centers to C and updates δ and σ against the new centers in parallel (Line 8), setting $\delta[i] \leftarrow d(p_i, c)^2$ and $\sigma[i] \leftarrow c$ whenever a new center c is closer to p_i than its current nearest. Since the batch sizes grow geometrically and sum to k, the schedule has $O(\log k)$ rounds.

Two points are worth highlighting. Because the centers within a round are independent and use the same frozen distribution, an entire batch is drawn in a single parallel pass; by contrast, k-means++ has k dependent sampling steps, since each newly chosen center changes the D^2 distribution used by the next step. Maintaining σ adds no asymptotic cost and yields the nearest-center assignment as a by-product, which we hand to the first Lloyd's iteration as a warm start (Sec. 4.2). In the implementation, duplicate centers are discarded and, if needed, the batch is topped up with farthest uncovered points so that the initializer returns exactly k distinct centers; this preserves the $O(nk)$ work bound and only enforces the interface expected by Lloyd's refinement.

3.2 Analysis

Theorem 3.1. *Doubling k-means computes k sampled centers on n points using $O(nk)$ work and $O(\log n \log k)$ span.*

Proof. We bound work and span separately.
Work. The geometric schedule runs $O(\log k)$ rounds. In round r, the sampling step builds the current D^2 distribution and draws b_r independent centers in $O(n + b_r \log n)$ work; the update step then compares every point against the b_r new centers and recomputes Φ with a Reduce, in $O(n\, b_r)$ work. The batch sizes sum to k centers, so $\sum_r b_r = k$ and the total update work is $O(n \sum_r b_r) = O(nk)$, which dominates the $O(n \log k + k \log n)$ sampling work.
Span. The $O(\log k)$ rounds are sequentially dependent, so it suffices to bound the span of one round. Computing Φ is a Reduce over the n points ($O(\log n)$ span); drawing the b_r centers from the resulting distribution has $O(\log n)$ span; and the update sets each $\delta[i]$ to the minimum over the b_r new centers, a per-point reduction of $O(\log b_r) \le O(\log k)$ span with all points processed in parallel (in Alg. 2, the inner loop of UpdateDistances is written as a sequential scan over S for clarity; computing the per-point minimum with a parallel reduce over S gives the stated span). A round thus has $O(\log n + \log k)$ span, and over $O(\log k)$ rounds the total is $O(\log n \log k)$. □

4 Ward's HAC Seeding and ANNS Refinement

This section presents the two remaining algorithmic components of UltraKMeans: a practical Ward's HAC seeder and an ANNS-accelerated Lloyd's refinement engine. They address different bottlenecks but use graph-based nearest-neighbor search at two complementary levels. The Ward's HAC seeder searches dynamically among evolving cluster representatives to avoid materializing all pairwise merge costs, whereas Lloyd's refinement searches a freshly built graph over the current centers to avoid scanning all k centers for every point.
Vamana graph search [62]. Vamana is a graph-based ANNS index that represents each indexed vector as a vertex in a bounded-degree directed graph. To answer a query, its beam search starts from a designated entry vertex, repeatedly expands the closest unvisited candidate in a bounded search list, and returns the closest vectors encountered. During

index construction, Vamana uses the same graph search to find candidate neighbors and robust pruning to bound vertex degree while retaining a navigable graph.

4.1 Practical Ward's HAC Seeding

Ward's HAC is a hierarchical agglomerative clustering (HAC) method closely related to the k-means objective. Starting from singleton clusters, it repeatedly merges the pair whose union causes the smallest increase in SSE. For clusters A and B, this increase simplifies as

$$D(A,B) := \frac{|A||B|}{|A|+|B|} \cdot \|\mu(A) - \mu(B)\|_2^2,$$

where $\mu(X)$ denotes the centroid of X. The direct algorithm maintains all pairwise costs and requires $\Theta(n^2)$ time, and any exact algorithm requires $\Omega(n^2)$ time under standard fine-grained complexity assumptions [2]. Abboud et al. [2] break this barrier for approximate Ward's HAC[2] by replacing the distance matrix with dynamic nearest-neighbor structures, but their theoretical realization uses dynamic LSH and geometrically increasing threshold rounds. Our goal is instead a practical, quality-oriented k-means initializer: it should avoid quadratic work on the full training set, return exactly k centers, and expose an assignment that Lloyd's refinement can reuse.

Our seeder has three phases; its pseudocode appears in Alg. 3. First, it selects a compact candidate set using D^2-style sampling, while maintaining a point-to-candidate assignment for every training point. It then replaces each candidate by the mean of its assigned points and gives the resulting representative the cluster weight equal to the corresponding number of assigned points. Second, weighted approximate Ward's HAC merges these representatives until k clusters remain, using weight-bucketed nearest-neighbor search to generate candidate merges and a lazy greedy queue to schedule them. Finally, the algorithm returns the centroids of the surviving clusters and composes the representative-level clustering with the recorded point-to-representative map, yielding a train-point assignment that warm-starts refinement.

Finding candidate merges. We partition clusters into geometric weight levels based on their weights (i.e., cluster sizes). For a fixed query cluster, the Ward weight multiplier varies by at most a factor of $1+\delta$ among candidates in the same level. Consequently, centroid distance provides a suitable proxy for generating candidates within each level. For a query cluster A, the algorithm searches the centroids in every level using squared Euclidean distance, evaluates the exact Ward cost of the candidate returned by each search, and keeps the least-cost candidate across levels. As shown by Abboud et al. [2], this multi-level approach yields an approximate nearest neighbor under Ward's linkage when the within-level searches satisfy their approximation guarantee.

[2]"approximate" here denotes a provable guarantee.

Scheduling merges. An eager scheduler would maintain the current nearest-neighbor candidate of every active cluster and refresh all candidates affected by each merge. We instead use a min-priority queue whose entry (d,u,v) records that the bucketed search returned v as an approximate candidate for source cluster u, with Ward cost d at the time the entry was created. The queue initially contains one such entry per active cluster. Because subsequent merges change cluster identities, centroids, and weights, an entry in the queue can become stale.

To handle this, when the minimum-key entry (d,u_0,v_0) is popped, the scheduler first maps both endpoints through union-find to the active clusters u and v that now contain them. If $u=v$, the pair has already collapsed into one cluster and is discarded. Otherwise, the scheduler recomputes the current cost $D(u,v)$ and merges the pair if $D(u,v) \le (1+\epsilon)d$. If this test fails, it performs a new bucketed query for u, obtaining a candidate v' with current cost d'. It merges (u,v') immediately if $d' \le (1+\epsilon)d$; otherwise, no merge occurs and a fresh candidate for u is inserted into the queue. After an accepted merge, the scheduler likewise queries the surviving cluster and inserts a fresh entry for it. All other stale entries remain untouched until they reach the front of the queue.

Abboud et al. [2] instead sweep a geometric sequence of global merge thresholds and, at each threshold, revisit the active clusters to merge those whose approximate-neighbor cost lies below it. Our scheduler is event-driven: it processes one queue entry at a time in increasing order of its stored key and uses the $(1+\epsilon)d$ test only to decide whether that entry still justifies an immediate merge. It is therefore not a literal implementation of their threshold-round algorithm, so we evaluate its quality empirically rather than claim their per-merge approximation guarantee.

4.2 ANNS-Accelerated Lloyd's Refinement

The assignment step of exact Lloyd's refinement compares every point with every center, performing $O(nk)$ work per iteration. Existing exact accelerations either maintain auxiliary bounds with irregular memory access or optimize the exhaustive search while retaining its linear dependence on k. Graph-based ANNS offers a different route, but applying it inside Lloyd's creates a recurring challenge: every centroid update changes the indexed points, so an index built for one iteration is stale in the next. Our goal is to reduce the per-point search without introducing cross-iteration dynamic-index maintenance, while retaining independent parallel assignment queries.

To avoid this cost, our refinement step uses the assignment from one iteration as search state for the next. Given the current point assignments, it first recomputes the k centroids and builds a Vamana graph over these updated centers. Beyond determining the updated centroids, the preceding assignments do not affect graph construction; they reappear only as starting points for the subsequent queries. It then

assigns all n points in parallel with multi-start beam search. For point p_i, the search begins from both the graph's global entry point and the center indexed by p_i's preceding assignment; the closest center found becomes the new assignment. We preserve center identifiers across the centroid update, so each preceding assignment still names a valid vertex in the freshly rebuilt graph. Consecutive Lloyd iterations are typically highly correlated, making that vertex a useful starting point that is often already near the new answer. The global entry point supplies an independent route through the graph when that warm start is poor.

4.3 Implementation Details

We implement UltraKMeans in C++ using ParlayLib [14] for fork-join parallelism and ParlayANN's Vamana implementation [51] for graph search.

Dynamic index for Ward's HAC. We maintain one Vamana graph for every nonempty weight level. When two clusters merge, we remove the absorbed representative using in-place deletion [66], then update the surviving representative's centroid and weight. If the survivor remains in the same level, we retain its existing adjacency list and use its updated coordinates in subsequent distance computations; if its weight crosses a level boundary, we delete it from the old graph and reinsert it into the new one. This lazy same-level update avoids reconstructing the survivor's adjacency after every merge; the local graph-repair and stale-edge-cleanup rules appear in appendix C.

Parallel execution. The Ward's HAC pipeline is partially parallelized: sampling, point assignment, representative construction, and portions of Vamana construction, search, and repair run in parallel, whereas the global priority queue selects one merge at a time and drives the corresponding index updates along a sequential merge path. For Lloyd's refinement, graph construction is parallel, and the n beam-search assignments are mutually independent and run in parallel.

5 Experiments

Experimental Setup. All experiments run on a 112-core machine (with 224 hyper-threads) with 4×2.1 GHz Intel Xeon Platinum 8176 CPUs (each with 38.5 MB L3 cache) and 1.5 TB of main memory. Unless otherwise stated, all experiments use all 224 hyper-threads with `numactl -i all` to interleave memory across sockets, and we report the mean of three trials preceded by one warmup trial. Note that the three trials average timing only. Our seeders are internally deterministic given a fixed random seed, so centers and SSE are identical across trials. Each run performs 25 Lloyd's refinement iterations, matching the default of faiss. By default, when $n > 256k$, we run seeding and refinement on a uniform sample of $256k$ points, which matches faiss's default cap of 256 training points per centroid, and compute the final SSE on the full dataset. Appendix D additionally reports a comparison

Dim.	ID	n	d	Name / notes
Low-d	**HT**	0.93M	10	HT sensor [39]
	HH	2.05M	7	Household [38]
	CH	4.21M	16	CHEM [27]
	GL	24.9M	3	GeoLifeNoScale [70]
	CM	321M	3	Cosmo50 [44]
	OSM	2.77B	2	OpenStreetMap [29]
High-d	**MN**	70.0K	784	MNIST [23]
	BD	84.6K	1024	Birds [30, 47]
	RD	420K	1024	Reddit [45, 56]
	ST	1.00M	128	SIFT [40]
	IC	1.28M	1024	ImageNet Convnet [22, 47]
	OA	5.00M	1536	OpenAI embeddings [57]

Table 2. Real-world datasets used in the overall, thread-scalability, and k-scalability experiments. We report the dataset ID, approximate number of points (n), and dimensionality (d); 'Dim.' denotes the low-/high-dimensional grouping. Synthetic SS-Varden workloads [28] are used only in the n-scalability study and are omitted here.

at k=1000 on nine datasets (excluding the three large-scale datasets), with every method trained on the full input rather than a subsample. For the experiments that vary k and n, however, we *disable* subsampling and run every method on the full input, since their purpose is to expose how cost and quality actually trend as the cluster count and input size grow. We test twelve real-world datasets spanning both low-dimensional ($d \leq 16$) and high-dimensional ($d \geq 128$) regimes, covering scales from 70 K to 2.77 B points (Tab. 2). When varying n, we use synthetic datasets from the Seed Spreader–Various Density (SS-Varden) generator [28], with $d = 8$ and n ranging from 1 M to 100 M. Unless stated otherwise, averages are geometric means and, like the GeoMean rows in Tab. 3, exclude OSM so that they are well-defined for all methods.

Tested Algorithms. We evaluate seven end-to-end k-means methods. Five are implemented within our framework and differ only in their seeding strategy: (1) *Doubling k-means* with a growth factor of $\alpha = 1.05$ (we ablate this choice in the supplementary material), (2) *Ward's HAC*, (3) *Random*: uniform random sampling, which serves as a trivial baseline, (4) *k-means++* [7], and (5) *k-means‖* [12]. For all internal methods, we use ANNS-based Lloyd's refinement, which warm-starts each query from its cluster assignment in the previous iteration. We also compare against two configurations of one external baseline: (6) *faiss-Random* [25], using Faiss's default random initialization, and (7) *faiss-AFK-MC^2* [10, 25], using the MCMC-based AFK-MC^2 approximation to k-means++ for initialization. Both use HNSW for point-to-center assignment as it is faster. Because AFK-MC^2 selects centers sequentially, we include it only in the overall comparison (Tab. 3) to show its quality; all scalability experiments use faiss-Random. Appendix D extends this comparison with two

Time: 1× 1.6× 2.5× 4.0× 6.3× ≥ 10× **SSE:** 1× 1.1× 1.2× 1.3× 1.4× ≥ 1.5×

			Our implementation										faiss			
			Doubling k-means		Ward's HAC		Random		k-means++		k-means‖		Random		AFK-MC^2	
		Dataset	Time	SSE	Time	SSE	Time	SSE	Time	SSE	Time	SSE	Time	SSE	Time	SSE
Low-dim.	$k=1000$	HT	0.618	6.67e+05	2.52	**5.90e+05**	**0.558**	2.00e+06	1.02	6.80e+05	1.12	6.72e+05	4.02	3.95e+06	4.61	6.82e+05
		HH	0.684	1.49e+06	3.14	**1.46e+06**	**0.655**	3.19e+06	0.844	1.48e+06	0.993	1.48e+06	3.89	3.55e+06	3.82	1.49e+06
		CH	0.619	1.79e+11	2.32	**1.69e+11**	**0.586**	2.99e+11	0.970	1.80e+11	1.08	1.78e+11	4.23	3.35e+11	5.31	2.15e+11
		GL	0.637	4.37e+09	2.72	**4.35e+09**	**0.613**	1.25e+12	0.823	4.37e+09	0.933	4.42e+09	4.43	2.16e+12	4.25	5.23e+09
		CM	0.672	6.74e+05	2.68	**6.70e+05**	**0.636**	6.81e+05	0.877	6.80e+05	1.00	6.75e+05	22.6	6.81e+05	22.8	6.76e+05
		OSM	0.678	1.17e+09	2.29	**1.06e+09**	**0.656**	4.11e+09	0.894	1.18e+09	1.37	1.17e+09	†	†	†	†
		Geo.	0.646	5.54e+07	2.67	**5.32e+07**	**0.608**	2.77e+08	0.903	5.57e+07	1.02	5.56e+07	5.81	3.69e+08	6.18	5.99e+07
	$k=10000$	HT	3.36	3.18e+04	16.9	3.74e+04	**2.39**	1.70e+05	12.2	3.16e+04	25.7	**3.15e+04**	15.0	1.67e+05	28.5	3.24e+04
		HH	5.34	3.20e+05	33.9	**3.10e+05**	**3.30**	7.60e+05	27.2	3.21e+05	55.2	3.19e+05	30.1	7.35e+05	43.3	3.21e+05
		CH	5.69	1.88e+10	25.6	**1.77e+10**	**4.01**	4.69e+10	33.8	1.88e+10	68.4	1.86e+10	37.7	4.58e+10	50.8	1.88e+10
		GL	6.04	1.81e+08	16.4	**1.78e+08**	**3.96**	6.94e+10	34.3	1.80e+08	66.7	1.81e+08	35.5	1.79e+11	41.4	3.06e+08
		CM	5.76	1.37e+05	35.7	**1.36e+05**	**4.30**	1.40e+05	33.8	1.37e+05	66.9	1.37e+05	56.1	1.41e+05	64.8	1.37e+05
		OSM	4.94	7.04e+07	28.5	**6.48e+07**	**3.10**	3.56e+08	32.6	7.05e+07	66.2	6.95e+07	†	†	†	†
		Geo.	5.13	5.44e+06	24.4	5.48e+06	**3.52**	3.58e+07	26.5	5.43e+06	53.4	**5.41e+06**	32.1	4.26e+07	44.2	6.07e+06
High-dim.	$k=1000$	MN	1.61	1.41e+06	13.4	**1.40e+06**	**1.27**	1.42e+06	2.22	1.41e+06	2.89	1.41e+06	3.46	1.41e+06	8.66	1.41e+06
		BD	1.97	1.68e+07	9.97	**1.63e+07**	**1.48**	1.68e+07	3.07	1.67e+07	4.08	1.67e+07	6.01	1.67e+07	12.6	1.67e+07
		RD	4.97	8.30e+04	28.7	**8.23e+04**	**4.16**	8.31e+04	9.48	8.31e+04	12.1	8.29e+04	18.1	8.30e+04	25.6	8.31e+04
		ST	1.35	5.11e+10	6.88	**5.10e+10**	**1.27**	5.11e+10	1.97	5.11e+10	2.57	5.11e+10	5.14	5.11e+10	5.66	5.11e+10
		IC	3.61	4.66e+08	19.6	**4.31e+08**	**3.28**	5.07e+08	8.30	4.75e+08	11.1	4.58e+08	29.2	4.92e+08	35.7	4.99e+08
		OA	8.25	1.13e+06	39.4	**1.13e+06**	**7.65**	1.13e+06	15.4	1.13e+06	19.4	1.13e+06	31.3	1.13e+06	43.9	1.13e+06
		Geo.	2.93	1.94e+07	16.5	**1.90e+07**	**2.51**	1.97e+07	5.03	1.94e+07	6.55	1.93e+07	11.0	1.95e+07	17.1	1.96e+07
	$k=10000$	MN	5.10	8.77e+05	117	**8.23e+05**	**4.40**	9.09e+05	15.0	8.77e+05	28.2	8.74e+05	7.76	9.03e+05	479	8.76e+05
		BD	5.79	1.04e+07	97.2	**9.69e+06**	**4.25**	1.11e+07	22.0	1.04e+07	39.9	1.04e+07	8.90	1.11e+07	794	1.04e+07
		RD	15.8	5.90e+04	313	**5.62e+04**	**9.46**	6.07e+04	99.1	5.90e+04	181	5.86e+04	149	5.99e+04	938	5.97e+04
		ST	8.73	4.06e+10	103	**4.04e+10**	**6.94**	4.06e+10	41.3	4.06e+10	68.4	4.05e+10	30.3	4.05e+10	106	4.05e+10
		IC	33.0	3.24e+08	111	3.26e+08	**12.5**	3.30e+08	296	3.25e+08	553	**3.22e+08**	365	3.23e+08	1107	3.22e+08
		OA	193	9.87e+05	592	**9.85e+05**	**125**	9.90e+05	1084	9.87e+05	2099	9.86e+05	1086	9.87e+05	Timeout	N/A
		Geo.	17.2	1.38e+07	170	**1.34e+07**	**11.2**	1.42e+07	87.1	1.38e+07	159	1.38e+07	70.6	1.41e+07	-	-

Table 3. Overall performance on real-world datasets with ANNS refinement. Lower is better. Bold marks each row's best time and SSE. † marks faiss-unsupported inputs ($n > 2^{31}$). Geo. indicates GeoMeans, which omit OSM to include all methods. The time limit is 3,600 seconds.

additional open-source implementations: CppKmeans [46] and Marigold [55]. For all methods, we report runtime (in seconds) and final sum of squared errors (SSE).

5.1 Overall Performance

Tab. 3 reports end-to-end runtime and final SSE under ANNS refinement at two representative cluster counts, k=1000 and k=10000. Across all datasets, Doubling k-means matches the final SSE of the k-means++ family to within a few percent on average and is faster than every method except Random. Ward's HAC achieves the best SSE in almost all cases (22/24), at the cost of increased runtime.

Runtime. Random is the fastest method. Its seeding cost is negligible, so its runtime is essentially the cost of ANNS refinement alone, a floor that is very hard for any seeder to beat. Doubling k-means comes next, adding only the small overhead of its geometric-batch D^2 sampling in $O(\log k)$ rounds. Its running time is within 1.2× that of Random at k=1000 and within 1.6× at k=10000. k-means++ and k-means‖ become markedly slower as k grows: the former makes k dependent D^2 picks, while the latter ends with sequential weighted-k-means++ downsampling. At k=1000, where refinement still dominates, this makes Doubling k-means only 1.4–1.7× faster than k-means++ and 1.6–2.2× faster than k-means‖. At k=10000 the seeding cost takes over and the margins grow to about 5.1× and 9.2–10.4× on average. The effect is pronounced on openai, one of our largest inputs (5M points, d=1536), where at k=10000 Doubling k-means is 5.6× faster than k-means++ and 10.9× faster than k-means‖, while staying within 1.6× that of Random. Ward's

HAC carries the highest seeding cost and is the slowest of our seeders at moderate k, the price of its quality advantage. On average, it is 4.4×–15.2× slower than Random. faiss-Random is slower than our implementations in most cases. This gap reflects differences in the underlying ANNS backends: faiss uses HNSW for assignment, whereas our methods use ParlayANN's Vamana index and warm-start each query from its cluster assignment in the previous Lloyd iteration. Across the tested dimensional regimes and cluster counts, faiss-Random is 4.4–9.5× slower than Random and 3.8–9.0× slower than Doubling k-means. faiss-AFK-MC^2 adds a sequential initializer; we include it primarily as a quality reference, and its additional cost becomes pronounced at k=10000.

Quality. In low dimensions, seeding quality is decisive. A poor random start strands the initial clusters in a local minimum that refinement cannot escape. Random therefore trails the smart seeders, typically by 2–5× in final SSE and more on cluster-structured inputs, reaching 287× worse than Doubling k-means on GeoLife. Doubling k-means itself closely tracks the k-means++ family, with average final-SSE gaps of about 2% and the largest gaps concentrated on HT and OSM. In high dimensions the effect vanishes. Distances concentrate, so many candidate centers look similarly from a typical point and Lloyd's refinement quickly pulls different initializations toward comparable fixed points [3]. As a result, all internal seeders and both faiss configurations land within a few percent of one another on most high-dimensional instances (Tab. 3). Ward's HAC attains the best quality of all methods, reaching the lowest SSE in 22 of the 24 instances (~2% below the next-best seeder on average), which justifies its higher seeding cost when quality is paramount. faiss-Random follows the same dimension-dependent pattern as Random. It is comparable on most high-dimensional datasets, though up to 14% behind on IC, but its low-dimensional geometric-mean SSE is 6–8× worse than that of Doubling k-means, driven largely by GeoLife. AFK-MC^2 largely closes this quality gap: its low-dimensional geometric-mean SSE is within 8–12% of Doubling k-means, and it is comparable on most high-dimensional datasets, at the cost of sequential initialization. Appendices E and F give the remaining supporting experiments: seeding-only results, the Doubling k-means growth-factor ablation, and the exact-versus-approximate HAC comparison for Ward's HAC.

5.2 Scaling

We study scalability along three axes, the input size n, the number of clusters k, and the thread count. As noted in the setup, the experiments that vary n and k disable subsampling so that the curves reflect each method's trend on increasing n and k. The main paper reports HT and ST; Fig. 9 to 11 gives the corresponding results on HH and IC.

Scalability: Input size. Fig. 2 varies n from 1M to 100M on the SS-Varden-8D synthetic workload at k=1000. All of our methods and faiss-Random scale close to linearly. A 100× increase in n raises our methods' runtime by 71–102× (and faiss-Random's by 114×). Doubling k-means and Random are the fastest and nearly identical, while k-means++ and k-means‖ run 1.7–3.4× slower because their seeding parallelizes less well. Ward's HAC grows the least, only 37×, because its hierarchical merging runs on a fixed D^2-sample of 5,000 points internal to the seeder (5k at k=1000) regardless of n; only its sampling and weighting passes scale with n, so refinement dominates its runtime at large n. faiss-Random is the slowest throughout, about 12× slower than Doubling k-means at the largest size. Quality shows a complementary effect. Random's seeding stays consistently worse than that of the smart seeders, but refinement closes most of the gap, and at the largest scale (100M) it brings Random's final SSE within 1.06× of the best. The trend is not monotone over all

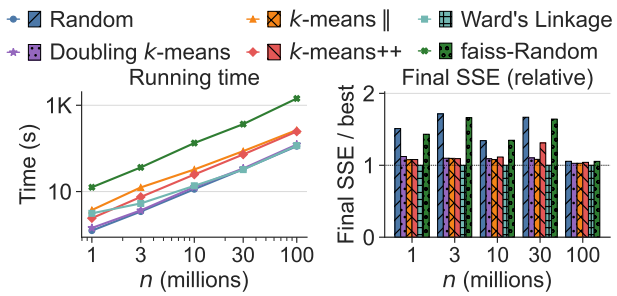


Figure 2. Running time and final SSE versus input size n on the SS-Varden-8D synthetic workload at k=1000 under ANNS refinement. SSE is relative to the best seeder.

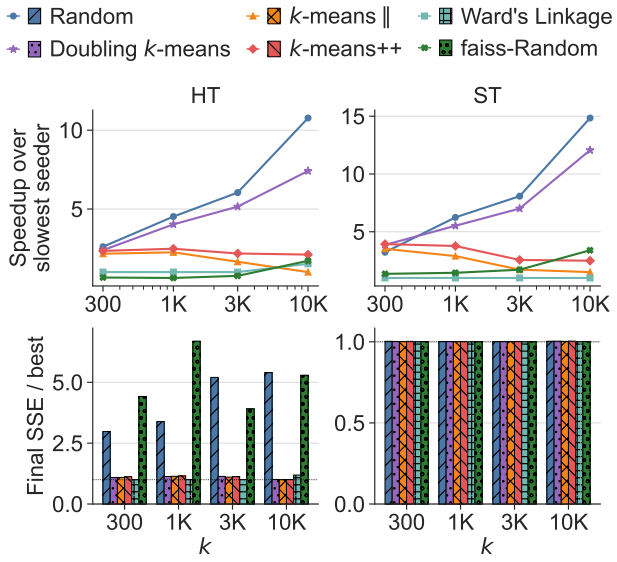


Figure 3. Speedup over the slowest seeder (top) and final SSE relative to the best seeder (bottom) versus k on HT and ST under ANNS refinement. Higher is better for speedup, lower for relative SSE.

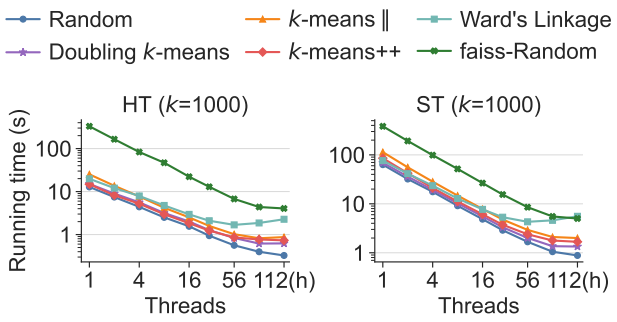


Figure 4. Total running time (seed + refinement) versus thread count on HT and ST at k=1000 under ANNS refinement. Both axes are log-scale; lower is better. (h) means hyperthreading is enabled.

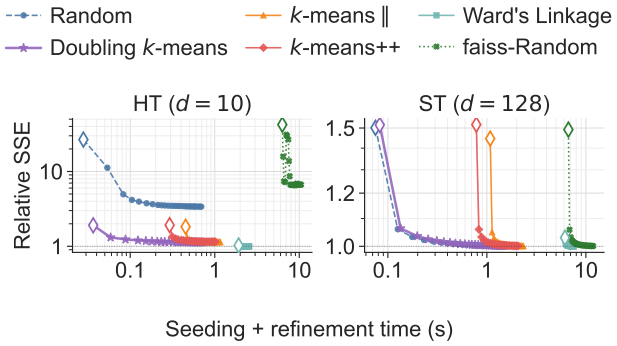


Figure 5. SSE versus cumulative seeding + refinement time over 25 Lloyd's iterations on HT and ST at k=1000, relative to the best final SSE per panel. Diamonds mark seeding completion. Lower and to the left is better.

intermediate sizes, but the largest instance shows that, with enough points per cluster, refinement can recover much of the gap even from a poor start.

Scalability: Number of clusters. Fig. 3 varies k from 300 to 10,000 on HT and ST, representative low- and high-dimensional datasets. The top row reports speedup over the slowest seeder at each k. Random and Doubling k-means pull away as k grows, since their seeding stays a small fraction of the total, while k-means++ and k-means‖ flatten near the bottom as their sequential seeding comes to dominate. The bottom row reports final SSE relative to the best seeder. On HT, Random and faiss-Random remain 1.9–6.7× worse than the smart seeders, whereas on ST the smart seeders all land within a few percent of the best. The cost ranking shifts with k but the quality ranking is set by dimension. Doubling k-means stays within a few percent of the k-means++ family across the whole range.

Scalability: Thread count. Fig. 4 reports total running time as the thread count grows to 224 (112 cores with hyperthreading) at k=1000. Across HT and ST, our four methods other than Ward's HAC achieve 20–72× end-to-end speedup from one to 224 threads, as their running time is dominated by the highly parallel ANNS refinement. Ward's HAC scales least, reaching only 8.7–14×, because its HAC merging is only partially parallelized. faiss-Random achieves 77–82× self-relative speedup because random seeding has negligible cost and most of its runtime is spent in the highly parallel ANNS phase. However, it remains slower than all of our methods on HT and all but Ward's HAC on ST with 224 threads.

5.3 Trade-off between Runtime and Quality

The results so far fix the budget at 25 Lloyd's iterations, which raises a natural question. Under a fixed time budget, is smart seeding worthwhile, or is it better to start from Random and spend the saved time on more refinement? Fig. 5 answers this by tracing each seeder's SSE against cumulative total time on one low-dimensional dataset (HT) and one high-dimensional dataset (ST) at k=1000, with a diamond marking the moment each seeder finishes and refinement begins. Note that Random's diamond does not sit at zero because it must first assign every training point to an initial center. This assignment dwarfs Random's essentially free seeding. Refinement converges quickly for every method: each curve realizes almost all of its improvement in the first few Lloyd's iterations and is essentially flat well before the 25-iteration budget is spent. What then determines the final SSE is where each curve starts and how low it settles, which is governed by dimension. On HT, the smart seeders begin near the floor and stay there, whereas Random starts higher and plateaus 2.2–3.4× above it. Its curve never crosses theirs, so for *any* time budget a smart seeder gives the lower SSE, and the time spent seeding well is repaid many times over. On ST, the curves converge to within a few percent of one another within the first few iterations, and because Random pays no seeding cost it reaches that common floor first. The frontier therefore belongs to Random at all but the smallest budgets, consistent with the near-flat objective in high dimensions, where almost every partition is a fixed point of Lloyd's update [3]. faiss-Random sits to the right of our methods throughout, reflecting its higher per-iteration cost, and in low dimensions also well above them, since its random seeding lands in the same poor basin as Random. The takeaway is that smart seeding is essential in low dimensions, where Lloyd's cannot recover from a poor start at any budget, and unnecessary in high dimensions, where refinement erases the difference within a few iterations regardless of budget.

6 Takeaways and Conclusions

In this paper, we presented UltraKMeans, an end-to-end shared-memory k-means system designed to scale with input size, dimensionality, cluster count, and thread count. In

addition to standard seeders, it introduces two complementary alternatives: Doubling k-means grows its D^2-sampling batch geometrically, returns exactly k centers, and removes the $\Theta(k^2)$ downsampling bottleneck of k-means‖ while empirically matching the final quality of the k-means++ family; our partially parallel Ward's HAC implementation brings this quality-oriented seeder into large-scale k-means pipelines.

Takeaways. Across twelve datasets spanning 70 K to 2.77 B points, Doubling k-means is the scalable choice for large n or k, while Ward's HAC attains the lowest final SSE on 22 of 24 instances at the cost of increased runtime. On the tested high-dimensional data, distance concentration makes Random often sufficient; on low-dimensional data, smart seeding remains decisive.

Future work. Open questions remain at both ends of the pipeline: fully parallelizing Ward's HAC requires new approaches to merge selection and dynamic-index maintenance, while incorporating quantization into ANNS-based Lloyd's could reduce memory traffic and search cost, especially for high-dimensional embeddings.

Acknowledgments

Multiple generative AI models were used to assist with code debugging, script development, and polishing the writing. The authors reviewed all AI-assisted output and take full responsibility for the resulting code and manuscript.

References

[1] 2026. RAPIDS cuML: GPU Machine Learning Algorithms. https://github.com/rapidsai/cuml.

[2] Amir Abboud, Vincent Cohen-Addad, and Hussein Houdrouge. 2019. Subquadratic high-dimensional hierarchical clustering. In *Proceedings of the 33rd International Conference on Neural Information Processing Systems*. Curran Associates Inc., Red Hook, NY, USA, Article 1039, 11 pages.

[3] Charu C Aggarwal, Alexander Hinneburg, and Daniel A Keim. 2001. On the surprising behavior of distance metrics in high dimensional space. In *International conference on database theory*. Springer, 420–434.

[4] Sara Ahmadian, Ashkan Norouzi-Fard, Ola Svensson, and Justin Ward. 2019. Better guarantees for k-means and euclidean k-median by primal-dual algorithms. *SIAM J. Comput.* 49, 4 (2019), FOCS17–97.

[5] Daniel Aloise, Amit Deshpande, Pierre Hansen, and Preyas Popat. 2009. NP-hardness of Euclidean sum-of-squares clustering. *Machine learning* 75 (2009), 245–248.

[6] Nimar S Arora, Robert D Blumofe, and C Greg Plaxton. 2001. Thread scheduling for multiprogrammed multiprocessors. *Theory of Computing Systems (TOCS)* 34, 2 (2001), 115–144.

[7] David Arthur and Sergei Vassilvitskii. 2006. *k-means++: The advantages of careful seeding*. Technical Report. Stanford.

[8] Anonymous Authors. 2026. Implementations for Ultra-KMeans. https://anonymous.4open.science/r/Ultra-KMeans-2CDD/.

[9] Ariful Azad, Georgios A Pavlopoulos, Christos A Ouzounis, Nikos C Kyrpides, and Aydin Buluç. 2018. HipMCL: a high-performance parallel implementation of the Markov clustering algorithm for large-scale networks. *Nucleic acids research* 46, 6 (2018), e33–e33.

[10] Olivier Bachem, Mario Lucic, Hamed Hassani, and Andreas Krause. 2016. Fast and provably good seedings for k-means. *Advances in Neural Information Processing Systems (NIPS)* 29 (2016).

[11] Olivier Bachem, Mario Lucic, and Andreas Krause. 2017. Distributed and provably good seedings for k-means in constant rounds. In *International Conference on Machine Learning*. PMLR, 292–300.

[12] Bahman Bahmani, Benjamin Moseley, Andrea Vattani, Ravi Kumar, and Sergei Vassilvitskii. 2012. Scalable K-Means+. *Proceedings of the VLDB Endowment* 5, 7 (2012).

[13] Lorenzo Beretta, Vincent Cohen-Addad, Silvio Lattanzi, and Nikos Parotsidis. 2023. Multi-swap k-means++. In *Advances in Neural Information Processing Systems (NIPS)*. 26069–26091.

[14] Guy E. Blelloch, Daniel Anderson, and Laxman Dhulipala. 2020. ParlayLib — a toolkit for parallel algorithms on shared-memory multicore machines. In *ACM Symposium on Parallelism in Algorithms and Architectures (SPAA)*. 507–509.

[15] Guy E. Blelloch, Jeremy T. Fineman, Phillip B. Gibbons, and Julian Shun. 2012. Internally deterministic parallel algorithms can be fast. In *ACM Symposium on Principles and Practice of Parallel Programming (PPOPP)*. 181–192.

[16] Guy E. Blelloch, Jeremy T. Fineman, Yan Gu, and Yihan Sun. 2020. Optimal parallel algorithms in the binary-forking model. In *ACM Symposium on Parallelism in Algorithms and Architectures (SPAA)*. 89–102.

[17] Guy E. Blelloch, Yan Gu, Julian Shun, and Yihan Sun. 2020. Randomized Incremental Convex Hull is Highly Parallel. In *ACM Symposium on Parallelism in Algorithms and Architectures (SPAA)*.

[18] Robert D. Blumofe and Charles E. Leiserson. 1998. Space-Efficient Scheduling of Multithreaded Computations. *SIAM J. on Computing* 27, 1 (1998).

[19] Adam Coates and Andrew Y Ng. 2012. Learning feature representations with k-means. In *Neural Networks: Tricks of the Trade: Second Edition*. Springer, 561–580.

[20] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. 2009. *Introduction to Algorithms (3rd edition)*. MIT Press.

[21] Ryan Ross Curtin. 2015. *Improving dual-tree algorithms*. Ph. D. Dissertation. Georgia Institute of Technology.

[22] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. 2009. Imagenet: A large-scale hierarchical image database. In *2009 IEEE conference on computer vision and pattern recognition*. Ieee, 248–255.

[23] Li Deng. 2012. The mnist database of handwritten digit images for machine learning research [best of the web]. *IEEE signal processing magazine* 29, 6 (2012), 141–142.

[24] Yufei Ding, Yue Zhao, Xipeng Shen, Madanlal Musuvathi, and Todd Mytkowicz. 2015. Yinyang k-means: A drop-in replacement of the classic k-means with consistent speedup. In *International conference on machine learning*. PMLR, 579–587.

[25] Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. 2025. The faiss library. *IEEE Transactions on Big Data* (2025).

[26] Charles Elkan. 2003. Using the triangle inequality to accelerate k-means. In *International Conference on Machine Learning (ICML)*. 147–153.

[27] Jordi Fonollosa, Sadique Sheik, Ramón Huerta, and Santiago Marco. 2015. Reservoir computing compensates slow response of chemosensor arrays exposed to fast varying gas concentrations in continuous monitoring. *Sensors and Actuators B: Chemical* 215 (2015), 618–629.

[28] Junhao Gan and Yufei Tao. 2017. On the hardness and approximation of Euclidean DBSCAN. *ACM Transactions on Database Systems (TODS)* 42, 3 (2017), 1–45.

[29] Geofabrik GmbH. 2024. Source of OSM data. https://download.geofabrik.de/.

[30] Gerry. 2023. Birds 525 Species - Image Classification. https://www.kaggle.com/datasets/gpiosenka/100-bird-species.

[31] Fabrizio Grandoni, Rafail Ostrovsky, Yuval Rabani, Leonard J Schulman, and Rakesh Venkat. 2022. A refined approximation for Euclidean k-means. *Inform. Process. Lett.* 176 (2022), 106251.

[32] Yan Gu, Zachary Napier, and Yihan Sun. 2022. Analysis of Work-Stealing and Parallel Cache Complexity. In *SIAM Symposium on Algorithmic Principles of Computer Systems (APOCS)*. SIAM, 46–60.

[33] Sudipto Guha, Rajeev Rastogi, and Kyuseok Shim. 1998. CURE: An efficient clustering algorithm for large databases. *ACM Sigmod record* 27, 2 (1998), 73–84.

[34] Mordechai Haklay and Patrick Weber. 2008. Openstreetmap: User-generated street maps. *IEEE Pervasive computing* 7, 4 (2008), 12–18.

[35] Greg Hamerly. 2010. Making k-means even faster. In *IEEE International Conference on Data Mining (ICDM)*. SIAM, 130–140.

[36] Sariel Har-Peled and Soham Mazumdar. 2004. On coresets for k-means and k-median clustering. In *ACM Symposium on Theory of Computing (STOC)*. 291–300.

[37] Simon Harris and Renato Cordeiro De Amorim. 2022. An extensive empirical comparison of k-means initialization algorithms. *Ieee Access* 10 (2022), 58752–58768.

[38] Georges Hebrail and Alice Berard. 2012. Individual household electric power consumption. UCI Machine Learning Repository. DOI: https://doi.org/10.24432/C58K54.

[39] Ramon Huerta, Thiago Mosqueiro, Jordi Fonollosa, Nikolai F Rulkov, and Irene Rodriguez-Lujan. 2016. Online decorrelation of humidity and temperature in chemical sensors for continuous monitoring. *Chemometrics and Intelligent Laboratory Systems* 157 (2016), 169–176.

[40] Herve Jegou, Matthijs Douze, and Cordelia Schmid. 2010. Product quantization for nearest neighbor search. *IEEE transactions on pattern analysis and machine intelligence* 33, 1 (2010), 117–128.

[41] Tapas Kanungo, David M Mount, Nathan S Netanyahu, Christine D Piatko, Ruth Silverman, and Angela Y Wu. 2002. An efficient k-means clustering algorithm: Analysis and implementation. *IEEE transactions on pattern analysis and machine intelligence* 24, 7 (2002), 881–892.

[42] Tapas Kanungo, David M Mount, Nathan S Netanyahu, Christine D Piatko, Ruth Silverman, and Angela Y Wu. 2002. A local search approximation algorithm for k-means clustering. In *ACM Symposium on Computational Geometry (SoCG)*. 10–18.

[43] Leonardo Kuffo, Sven Hepkema, and Peter Boncz. 2026. A Super Fast K-means for Indexing Vector Embeddings. *arXiv preprint arXiv:2603.20009* (2026).

[44] YongChul Kwon, Dylan Nunley, Jeffrey P Gardner, Magdalena Balazinska, Bill Howe, and Sarah Loebman. 2010. Scalable clustering algorithm for N-body simulations in a shared-nothing cluster. In *International Conference on Scientific and Statistical Database Management*. Springer, 132–150.

[45] Zehan Li, Xin Zhang, Yanzhao Zhang, Dingkun Long, Pengjun Xie, and Meishan Zhang. 2023. Towards general text embeddings with multi-stage contrastive learning. *arXiv preprint arXiv:2308.03281* (2023).

[46] libscran. 2026. kmeans: A C++ library for k-means. https://github.com/libscran/kmeans.

[47] Zhuang Liu, Hanzi Mao, Chao-Yuan Wu, Christoph Feichtenhofer, Trevor Darrell, and Saining Xie. 2022. A convnet for the 2020s. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 11976–11986.

[48] Stuart Lloyd. 1982. Least squares quantization in PCM. *IEEE transactions on information theory* 28, 2 (1982), 129–137.

[49] Clemens Lutz, Sebastian Breß, Tilmann Rabl, Steffen Zeuch, and Volker Markl. 2018. Efficient k-means on GPUs. In *Proceedings of the 14th International Workshop on Data Management on New Hardware*. 1–3.

[50] Konstantin Makarychev, Aravind Reddy, and Liren Shan. 2020. Improved guarantees for k-means++ and k-means++ parallel. *Advances in Neural Information Processing Systems (NIPS)* 33 (2020), 16142–16152.

[51] Magdalen Dobson Manohar, Zheqi Shen, Guy Blelloch, Laxman Dhulipala, Yan Gu, Harsha Vardhan Simhadri, and Yihan Sun. 2024. ParlayANN: Scalable and Deterministic Parallel Graph-Based Approximate Nearest Neighbor Search Algorithms. In *Proceedings of the 29th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming* (Edinburgh, United Kingdom) *(PPoPP '24)*. Association for Computing Machinery, New York, NY, USA, 270–285. doi:10.1145/3627535.3638475

[52] James B McQueen. 1967. Some methods of classification and analysis of multivariate observations. In *Proc. of 5th Berkeley Symposium on Math. Stat. and Prob.* 281–297.

[53] Xiangrui Meng, Joseph Bradley, Burak Yavuz, Evan Sparks, Shivaram Venkataraman, Davies Liu, Jeremy Freeman, DB Tsai, Manish Amde, Sean Owen, Doris Xin, Reynold Xin, Michael J Franklin, Reza Zadeh, Matei Zaharia, and Ameet Talwalkar. 2016. MLlib: Machine learning in Apache Spark. *Journal of Machine Learning Research* 17, 34 (2016), 1–7.

[54] Disa Mhembere, Da Zheng, Carey E Priebe, Joshua T Vogelstein, and Randal Burns. 2017. knor: a NUMA-optimized in-memory, distributed and semi-external-memory k-means library. In *Proceedings of the 26th International Symposium on High-Performance Parallel and Distributed Computing*. 67–78.

[55] Kasper Overgaard Mortensen, Fatemeh Zardbani, Mohammad Ahsanul Haque, Steinn Ymir Agustsson, Davide Mottin, Philip Hofmann, and Panagiotis Karras. 2023. Marigold: efficient k-means clustering in high dimensions. *Proceedings of the VLDB Endowment (PVLDB)* 16, 7 (2023), 1740–1748.

[56] Niklas Muennighoff, Nouamane Tazi, Loïc Magne, and Nils Reimers. 2023. Mteb: Massive text embedding benchmark. In *Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics*. 2014–2037.

[57] Arvind Neelakantan, Tao Xu, Raul Puri, Alec Radford, Jesse Michael Han, Jerry Tworek, Qiming Yuan, Nikolas Tezak, Jong Wook Kim, Chris Hallacy, Johannes Heidecke, Pranav Shyam, Boris Power, Tyna Eloundou Nekoul, Girish Sastry, Gretchen Krueger, David Schnurr, Felipe Petroski Such, Kenny Hsu, Madeleine Thompson, Tabarak Khan, Toki Sherbakov, Joanne Jang, Peter Welinder, and Lilian Weng. 2022. Text and code embeddings by contrastive pre-training. *arXiv preprint arXiv:2201.10005* (2022).

[58] Zhen Peng, Minjia Zhang, Kai Li, Ruoming Jin, and Bin Ren. 2023. iqan: Fast and accurate vector search with efficient intra-query parallelism on multi-core architectures. In *Proceedings of the 28th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming*. 313–328.

[59] N. Scoville, H. Aussel, M. Brusa, P. Capak, C. M. Carollo, M. Elvis, M. Giavalisco, L. Guzzo, G. Hasinger, C. Impey, J.-P. Kneib, O. LeFevre, S. J. Lilly, B. Mobasher, A. Renzini, R. M. Rich, D. B. Sanders, E. Schinnerer, D. Schminovich, P. Shopbell, Y. Taniguchi, and N. D. Tyson. 2007. The cosmic evolution survey (COSMOS): overview. *The Astrophysical Journal Supplement Series* 172, 1 (2007), 1.

[60] David Sculley. 2010. Web-scale k-means clustering. In *Proceedings of the 19th international conference on World wide web*. 1177–1178.

[61] Jack Spalding-Jamieson, Eliot Wong Robson, and Da Wei Zheng. 2025. Scalable k-means clustering for large k via seeded approximate nearest-neighbor search. *arXiv preprint arXiv:2502.06163* (2025).

[62] Suhas Jayaram Subramanya, Devvrit, Rohan Kadekodi, Ravishankar Krishaswamy, and Harsha Vardhan Simhadri. 2019. DiskANN: fast accurate billion-point nearest neighbor search on a single node. In *Proceedings of the 33rd International Conference on Neural Information Processing Systems*. Curran Associates Inc., Red Hook, NY, USA, Article 1233, 11 pages.

[63] Leyuan Wang, Sean Baxter, and John D Owens. 2016. Fast parallel skew and prefix-doubling suffix array construction on the GPU. *Concurrency and Computation: Practice and Experience* 28, 12 (2016), 3466–3484.

[64] Letong Wang, Xiaojun Dong, Yan Gu, and Yihan Sun. 2023. Parallel Strong Connectivity Based on Faster Reachability. *ACM SIGMOD International Conference on Management of Data (SIGMOD)* 1, 2 (2023), 1–29.

[65] Junjie Wu. 2012. Cluster analysis and K-means clustering: an introduction. In *Advances in K-Means clustering: A data mining thinking*. Springer, 1–16.

[66] Haike Xu, Magdalen Dobson Manohar, Philip A. Bernstein, Badrish Chandramouli, Richard Wen, and Harsha Vardhan Simhadri. 2025. In-Place Updates of a Graph Index for Streaming Approximate Nearest Neighbor Search. arXiv:2502.13826 [cs.IR]

[67] Lijun Zhang, Hui Guan, Yufei Ding, Xipeng Shen, and Hamid Krim. 2021. Reuse-centric k-means configuration. *Information Systems* 100 (2021), 101787.

[68] Tian Zhang, Raghu Ramakrishnan, and Miron Livny. 1996. BIRCH: an efficient data clustering method for very large databases. *ACM sigmod record* 25, 2 (1996), 103–114.

[69] Weizhong Zhao, Huifang Ma, and Qing He. 2009. Parallel k-means clustering based on MapReduce. In *International Conference on Cloud Computing (CloudCom)*. Springer, 674–679.

[70] Yu Zheng, Like Liu, Longhao Wang, and Xing Xie. 2008. Learning transportation mode from raw gps data for geographic applications on the web. In *International World Wide Web Conference (WWW)*. 247–256.

This supplement follows the order in which the main paper references the additional material. We first expand the related work, give pseudocode and implementation details for the Ward's HAC seeder, and then report the additional experimental results: the full-data comparison, seeding-only results, ablations, and the remaining scalability and runtime–quality plots.

A More Related Work

***k*-means with Theoretical Guarantees.** Because minimizing the k-means objective is NP-hard [5], most theoretical work focuses on algorithms with provable approximation guarantees. k-means++ [7] achieves an $O(\log k)$-competitive ratio in expectation via D^2-sampling, while local search and its variants [4, 13, 31, 42] provide constant-factor guarantees (e.g., 6.357 or 9) by iteratively swapping centers to reduce cost. Recent analyses further tighten the guarantees of k-means++ and k-means|| [50] and characterize the tradeoff between k-means||'s round count and its seeding quality [11]. However, local search algorithms are less commonly adopted in practice because they require exhaustive neighborhood exploration with substantially higher computational complexity, and are therefore beyond the scope of this paper.

Refinement and Libraries. Several refinement methods use triangle-inequality bounds to prune point–center comparisons, including Elkan [26] and Hamerly [35] (Sec. 2). Yinyang k-means [24] maintains group-level distance bounds to skip most remaining comparisons. Mini-batch methods instead update centroids from small random batches, trading final SSE for a per-iteration cost independent of n [60]. These designs are packaged in widely used implementations, including faiss [25], CppKmeans [46], and the Marigold reference code [55]; we compare against all of them empirically in Sec. 5 and appendix D. Other systems reuse work across k-means configurations [67] or optimize graph-based vector search on multicore architectures [58].

Scalable Hierarchical Clustering and Coresets. Classical systems such as BIRCH [68] and CURE [33] scale hierarchical clustering by merging compact summaries or samples of the data rather than all points, the same strategy our sampled Ward's HAC seeder adopts (Sec. 4); Abboud et al. [2] provide the subquadratic approximate Ward's HAC framework we build on. Coreset constructions [36] instead compress the input into a small weighted point set on which any k-means solver runs with bounded quality loss, a principled counterpart to the uniform training subsampling we adopt from faiss (Sec. 5).

GPU-based and Distributed Algorithms. Several systems target k-means on GPUs or in distributed settings. GPU implementations [1, 49] typically parallelize the assignment step by broadcasting centers to all streaming multiprocessors and computing distances in bulk, achieving high throughput on moderate cluster counts. In the distributed setting, frameworks such as MapReduce-based k-means [69] and Spark MLlib [53] partition the dataset across nodes, perform local assignment and centroid accumulation, and synchronize via a global reduce step. Distributed graph-clustering systems likewise exploit sparse-matrix primitives at scale [9]. Our work is complementary: we focus on shared-memory multicore parallelism, exploiting cache locality, work-stealing, and ANNS-based assignment to scale across all four axes (n, d, k, and thread count) on a single machine.

B Ward's HAC Seeder Pseudocode

Alg. 3 gives pseudocode for the Ward's HAC seeder described in Sec. 4.

Sampling and assignment handoff (Lines 1–7). The seeder sets the target sample size to $\min(\lceil samw \cdot k \rceil, n)$ and runs the five-round D^2 sampler while maintaining a point-to-representative map ρ (Lines 1–2). It uses each assigned group to form a weighted representative (Line 3), applies weighted Ward's HAC to reduce these representatives to k active clusters (Line 4), materializes their current centroids as C (Line 5), and composes the two assignment maps to warm-start Lloyd's refinement (Line 7).

Bucketed candidate generation (Lines 11–17). The HAC routine places each active representative in a geometric weight level with its own Vamana index. For every active cluster u, it queries all nonempty levels by centroid distance and evaluates each returned candidate by Ward cost. These searches run in parallel across clusters; their least-cost candidates initialize the global heap in bulk.

Lazy merge scheduling (Lines 19–33). Heap entries can become stale after merges, so the algorithm maps both endpoints to their current union-find representatives and discards self-pairs. It recomputes the Ward cost and accepts a

Algorithm 3: Ward's HAC Seeder

Input: Point set P, number of clusters k, sample factor $samw$, approximation parameters ϵ, δ
Output: Initial centers C, assignments σ

1 $s \leftarrow \min(\lceil samw \cdot k \rceil,\ |P|)$
2 $(S, \rho) \leftarrow$ `D2SampleAndAssign`(P, s) *// ρ is the assignment maintained by the sampler*
3 $(\widehat{S}, w) \leftarrow$ `WeightedRepresentatives`(P, S, ρ) *// replace each candidate by its assigned mean*
4 $(C_S, \pi) \leftarrow$ `ApproxWardHAC`$(\widehat{S}, w, k, \epsilon, \delta)$
5 $C \leftarrow \{\widehat{s}_j : j \in C_S\}$
6 **foreach** $p_i \in P$ **do** *// in parallel*
7 $\quad \sigma[i] \leftarrow \pi[\rho[i]]$ *// compose point-to-sample and sample-to-cluster maps*
8 **return** C, σ
9 **Function** `ApproxWardHAC`$(X, w, k, \epsilon, \delta)$**:**
10 $\quad$ Initialize one active cluster per point in X
11 $\quad$ Place each cluster u in weight level $\lfloor \log_{1+\delta} w_u \rfloor$
12 $\quad$ Build one dynamic Vamana index per nonempty level
13 $\quad$ Initialize an array h indexed by active clusters
14 $\quad$ **foreach** *active cluster u* **do** *// in parallel*
15 $\quad\quad$ $(v, d) \leftarrow$ `NearestByBucketedANNS`(u) *// query all weight levels and keep the best Ward cost*
16 $\quad\quad$ $h[u] \leftarrow (d, u, v)$
17 $\quad$ $H \leftarrow$ `MinHeap`(h)
18 $\quad$ **while** *more than k clusters remain* **do**
19 $\quad\quad$ $(d, u_0, v_0) \leftarrow \textsc{PopMin}(H)$; $\quad u \leftarrow \textsc{Find}(u_0)$; $\quad v \leftarrow \textsc{Find}(v_0)$
20 $\quad\quad$ **if** $u = v$ **then**
21 $\quad\quad\quad$ continue
22 $\quad\quad$ $d' \leftarrow D(u, v)$
23 $\quad\quad$ **if** $d' \le (1+\epsilon)d$ **then**
24 $\quad\quad\quad$ $z \leftarrow$ `MergeClusters`(u, v) *// update centroid, weight, and dynamic indices*
25 $\quad\quad$ **else**
26 $\quad\quad\quad$ $(v', d'') \leftarrow$ `NearestByBucketedANNS`(u)
27 $\quad\quad\quad$ **if** $d'' \le (1+\epsilon)d$ **then**
28 $\quad\quad\quad\quad$ $z \leftarrow$ `MergeClusters`(u, v')
29 $\quad\quad\quad$ **else**
30 $\quad\quad\quad\quad$ $z \leftarrow u$
31 $\quad\quad$ $(y, d_y) \leftarrow$ `NearestByBucketedANNS`(z)
32 $\quad\quad$ **if** $y \neq z$ **then**
33 $\quad\quad\quad$ Push (d_y, z, y) into H
34 $\quad$ **return** *active representative indices and sample assignments*

pair whose cost remains within a factor $1+\epsilon$ of its stored key (Lines 22–24). Otherwise, it refreshes the first endpoint with another bucketed query (Line 26). Finally, it inserts a candidate for the surviving cluster (Line 33), avoiding eager updates to all affected entries.

C Additional Implementation Details

We describe additional implementation optimizations, robustness details, and Ward's HAC-specific implementation choices omitted from the main paper.

Empty Cluster Recovery. During centroid recomputation, some clusters may lose all assigned points, particularly in early iterations with poor initialization. We re-seed each empty cluster from a random point of a non-empty *donor* cluster, sampling without replacement and rotating round-robin through the largest donors so the re-seeds are drawn from diverse regions of the data (renormalizing the new centroid for non-metric distances). In the degenerate case where no donor has a spare point (e.g., $n \le k$), we instead sample from the full dataset. This gives every empty cluster a valid replacement centroid before the next assignment step while avoiding identical re-seeds; the subsequent assignment determines whether the cluster becomes populated.

Batched Center-Index Construction. The implementation rebuilds the center index for ANNS refinement with ParlayANN's batch-insertion interface and a large batch fraction, processing the insertions within each batch in parallel. The preceding point assignments determine the updated centroid coordinates and later serve as query start points, but they are not otherwise used to construct the graph.

Ward sampling. Our default Ward's HAC seeder requests $\min(\lceil samw \cdot k \rceil, n)$ candidates from a five-round D^2 sampler and uses $samw$=5 in the main experiments. The sampler updates point-to-representative assignments as it runs, and we reuse these assignments when constructing the weighted representatives rather than performing a second ANNS pass. After HAC, composing the representative-level assignment with this map supplies the seeding-to-refinement warm start described in Sec. 4.2.

Dynamic index maintenance. Let W be the total weight. We maintain $\lceil \log_{1+\delta} W \rceil + 1$ levels and place a cluster of weight s in level $\lfloor \log_{1+\delta} s \rfloor$. Each nonempty level maintains its own Vamana graph. For small levels, we scan the active representatives directly; larger levels use Vamana search. When two clusters merge, the absorbed representative is removed using the in-place deletion approach of Xu et al. [66]. A beam search around the deleted vertex supplies a small pool of active replacement candidates and identifies a subset of its incoming neighbors. For each identified incoming neighbor, we replace the deleted edge with edges to nearby candidates from this pool; we also connect active outgoing neighbors through the replacement candidates. Incoming edges not encountered by the deletion search may remain stale.

Our implementation removes these remaining stale edges opportunistically rather than through periodic consolidation. Whenever a greedy search visits a vertex, it filters inactive vertices from that vertex's adjacency list before continuing the traversal and writes back the filtered list. The surviving cluster's centroid and weight are updated in place. If it remains in the same level, its adjacency list is retained and subsequent searches evaluate distances using the updated coordinates; if its weight crosses a level boundary, the representative is deleted from its old graph and inserted into the graph for its new level.

D Comparison with Other Implementations

We benchmark two additional open-source k-means implementations on the same machine and datasets as Sec. 5: (1) *CppKmeans* [46] (libscran), a parallel C++ library that combines random, k-means++, or variance-partition seeding with Lloyd's, Hartigan–Wong, or mini-batch [60] refinement; and (2) the reference implementation of *Marigold* [55], which also provides serial accelerated exact variants (Elkan [26], Hamerly [35], and stepwise). All Lloyd's-based methods run for 25 refinement iterations. Hartigan–Wong instead uses its own convergence criterion, taking 10–16 sweeps here.

For a direct comparison, every configuration in Tab. 4 disables subsampling and trains on the full input; SSE is likewise evaluated on the full data. The table reports k=1000 on nine datasets (CM, OSM, and OA are omitted). The no-subsampling faiss sweep covers all nine reported datasets. Marigold's reference driver requires square dimensionality and enforces $nk < 2^{31} - 1$; CH, GL, and ST therefore report N/A, while its feasible RD and IC runs timed out.

Three observations stand out. First, Ward's HAC attains the lowest SSE on eight of the nine datasets under the common full-data protocol. The only exception is MN, where CppKmeans's Hartigan–Wong configuration is 0.66% lower. Second, Doubling k-means remains substantially faster: it is 1.28–13.3× faster than Ward's HAC (3.38× geometric mean), 1.84–25.7× faster than CppKmeans's k-means++Lloyd configuration across all nine rows (7.11× geometric mean), and 6.14–13.5× faster than faiss-AFK-MC^2 across all nine faiss rows (9.43× geometric mean). Third, full-data training materially changes the low-dimensional comparison. On GL, for example, Ward's HAC reaches 6.95×10^8 SSE, ahead of Hartigan–Wong's 7.39×10^8 and Doubling k-means's 9.42×10^8; the previous gap was therefore largely a training-protocol effect rather than an algorithm-only difference.

E Additional Seeding Results

Seeding time and initialization quality. Tab. 5 and 6 isolate the five internal seeders before Lloyd's refinement, reporting seeding time and the SSE of the initial clustering at the two cluster counts used in the main overall comparison. These tables separate the cost and quality of initialization from the subsequent ANNS refinement. These experiments intentionally follow the main paper's default training protocol, including the uniform $256k$ training subsample when $n > 256k$, and are separate from the no-subsampling comparison in Tab. 4.

Seeder thread scalability. Fig. 6 reports the self-relative speedup of the seeding phase on the four datasets used in the end-to-end thread-scaling study. Random is omitted because its seeding time is below 0.01 seconds at every thread count, and faiss is omitted because it does not report a seeding/refinement split.

F Additional Ablations

Growth-factor sensitivity. Fig. 7 varies the geometric growth factor α in Doubling k-means and reports its effect on seeding time, initialization SSE, and final SSE. The marked default α=1.05 is the setting used throughout the main experiments.

Exact versus approximate HAC. Fig. 8 compares exact and approximate HAC within the sampled Ward's HAC seeder over several HAC sample sizes. This isolates the runtime and quality effects of approximate candidate generation from those of the preceding sampling stage.

G Additional Scaling and Trade-off Results

The following plots give the HH and IC counterparts to the HT and ST results in the main paper, in the same order as the corresponding discussion there.

Number of clusters. Fig. 9 varies k on HH and IC. As in the main-paper plot, the top panels report speedup over the slowest seeder at each k, while the bottom panels report final SSE relative to the best seeder.

Thread count. Fig. 10 reports end-to-end running time versus thread count on HH and IC at k=1000, complementing the HT and ST curves in the main paper.

Time: 1× 1.6× 2.5× 4.0× 6.3× ≥ 10× **SSE:** 1× 1.1× 1.2× 1.3× 1.4× ≥ 1.5×

		UltraKMeans (ours)				faiss				CppKmeans								Marigold	
		Doubling k-means		Ward's HAC		Random		AFK-MC^2		k-means++Lloyd		Rand+Lloyd		k-means+++H–W		k-means+++MB		Elkan (serial)	
	Dataset	**Time**	**SSE**	**Time**	**SSE**	**Time**	**SSE**	**Time**	**SSE**	**Time**	**SSE**	**Time**	**SSE**	**Time**	**SSE**	**Time**	**SSE**	**Time**	**SSE**
Low-dim.	HT	1.49	7.07e+05	3.17	**5.77e+05**	12.9	1.94e+06	13.9	6.54e+05	2.73	6.65e+05	**0.750**	1.79e+06	31.4	6.60e+05	2.16	7.45e+05	71.9	1.42e+07
	HH	2.50	1.41e+06	5.06	**1.38e+06**	28.2	2.73e+06	33.7	1.41e+06	5.33	1.42e+06	**1.16**	3.09e+06	255	1.40e+06	4.69	1.65e+06	163	2.42e+06
	CH	4.75	1.73e+11	7.06	**1.64e+11**	54.3	2.91e+11	52.8	1.70e+11	16.0	1.71e+11	**2.78**	2.94e+11	658	1.68e+11	12.9	1.94e+11	N/A	N/A
	GL	27.8	9.42e+08	35.4	**6.95e+08**	295	4.29e+12	292	8.90e+08	61.8	7.47e+08	**5.75**	1.29e+12	666	7.39e+08	55.9	9.11e+08	N/A	N/A
High-dim.	MN	**0.861**	1.41e+06	11.4	1.40e+06	3.45	1.41e+06	8.72	1.41e+06	12.4	1.41e+06	11.1	1.41e+06	521	**1.40e+06**	1.81	1.57e+06	117	1.42e+06
	BD	**0.964**	1.68e+07	9.33	**1.63e+07**	5.80	1.67e+07	12.9	1.67e+07	23.8	1.67e+07	18.5	1.67e+07	498	1.65e+07	6.32	1.87e+07	200	1.81e+07
	RD	**5.87**	8.26e+04	33.8	**8.22e+04**	28.2	8.27e+04	36.0	8.26e+04	110	8.28e+04	95.4	8.27e+04	Timeout	N/A	19.3	9.49e+04	Timeout	N/A
	ST	**3.57**	5.07e+10	12.5	**5.06e+10**	17.7	5.07e+10	22.3	5.06e+10	32.9	5.07e+10	18.2	5.07e+10	Timeout	N/A	12.6	5.68e+10	N/A	N/A
	IC	**12.7**	4.69e+08	34.7	**4.31e+08**	93.0	4.75e+08	98.5	4.85e+08	327	4.57e+08	292	4.56e+08	Timeout	N/A	41.0	5.67e+08	Timeout	N/A

Table 4. Full-data comparison with additional open-source k-means implementations at k=1000. Every configuration trains on the full input with subsampling disabled. Time is seeding plus refinement in seconds and excludes input I/O. SSE is evaluated on the full data. Bold marks the best completed value per dataset and metric. H–W is Hartigan–Wong and MB is mini-batch. The Marigold reference code is single-threaded. Timeout marks runs that did not finish within 1800 seconds. N/A means not applicable.

Time: 1× 1.6× 2.5× 4.0× 6.3× ≥ 10× **SSE:** 1× 1.1× 1.3× 1.5× 1.7× ≥ 2.0×

			Doubling k-means		Ward's HAC		Random		k-means++		k-means‖		
		Dataset	**Time**	**SSE**	**Time**	**SSE**	**Time**	**SSE**	**Time**	**SSE**	**Time**	**SSE**	**Best Final SSE**
Real-world	*Low-dim.*	HT	0.042 (7%)	1.91×	1.99 (79%)	1.03×	<0.001 (0%)	26.7×	0.337 (33%)	1.92×	0.459 (41%)	1.83×	5.90e+05
		HH	0.044 (6%)	1.42×	2.53 (80%)	1.04×	<0.001 (0%)	7.26×	0.281 (33%)	1.41×	0.414 (42%)	1.37×	1.46e+06
		CH	0.032 (5%)	1.66×	1.78 (76%)	1.04×	<0.001 (0%)	6.14×	0.298 (31%)	1.69×	0.426 (39%)	1.60×	1.69e+11
		GL	0.031 (5%)	1.07×	2.11 (78%)	1.00×	<0.001 (0%)	1451×	0.280 (34%)	1.06×	0.365 (39%)	1.07×	4.35e+09
		CM	0.034 (5%)	1.39×	2.05 (76%)	1.06×	<0.001 (0%)	1.65×	0.284 (32%)	1.39×	0.384 (38%)	1.35×	6.70e+05
		OSM	0.036 (5%)	1.64×	1.80 (79%)	1.04×	<0.001 (0%)	18.0×	0.296 (33%)	1.73×	0.643 (47%)	1.61×	1.06e+09
		GeoMean	0.036 (6%)	1.49×	2.03 (78%)	1.04×	<0.001 (0%)	19.3×	0.296 (33%)	1.51×	0.440 (41%)	1.45×	8.76e+07
	High-dim.	MN	0.140 (9%)	1.61×	12.1 (91%)	1.04×	0.003 (0%)	1.62×	1.06 (48%)	1.61×	1.74 (60%)	1.55×	1.40e+06
		BD	0.205 (10%)	1.88×	8.57 (86%)	1.02×	0.003 (0%)	1.97×	1.78 (58%)	1.87×	2.78 (68%)	1.70×	1.63e+07
		RD	0.598 (12%)	1.79×	24.5 (86%)	1.03×	0.002 (0%)	1.79×	5.42 (57%)	1.79×	8.13 (67%)	1.74×	8.23e+04
		ST	0.087 (6%)	1.52×	5.68 (83%)	1.03×	<0.001 (0%)	1.50×	0.796 (40%)	1.52×	1.37 (53%)	1.45×	5.10e+10
		IC	0.596 (17%)	2.15×	16.5 (84%)	1.06×	0.001 (0%)	2.28×	5.41 (65%)	2.20×	8.08 (73%)	1.86×	4.31e+08
		OA	0.887 (11%)	1.79×	31.9 (81%)	1.02×	<0.001 (0%)	1.79×	8.01 (52%)	1.80×	12.1 (62%)	1.74×	1.13e+06
		GeoMean	0.304 (10%)	1.78×	14.0 (85%)	1.03×	0.001 (0%)	1.81×	2.66 (53%)	1.78×	4.17 (64%)	1.67×	1.90e+07

Table 5. Seeding-only running time and initialization SSE at k = 1000. The SSE columns report initialization SSE *relative to the best final SSE* (last column) achieved after refinement across all seeding methods; lower (closer to 1×) is better. Parenthesized percentages next to times are seeding's share of seed+refinement time; underlining marks the best value in each row. The time color scale excludes Random, whose near-zero seed time would otherwise dominate the baseline.

		Time:	1×	1.6×	2.5×	4.0×	6.3×	≥ 10×	SSE:	1×	1.1×	1.3×	1.5×	1.7×	≥ 2.0×	
			Doubling k-means		Ward's HAC		Random		k-means++		k-means‖					
		Dataset	**Time**	**SSE**	**Time**	**SSE**	**Time**	**SSE**	**Time**	**SSE**	**Time**	**SSE**	**Best Final SSE**			
Real-world	*Low-dim.*	HT	0.866 (26%)	1.69×	14.6 (87%)	1.28×	0.003 (0%)	20.8×	9.57 (78%)	1.67×	23.2 (90%)	1.62×	3.15e+04			
		HH	1.45 (27%)	1.55×	30.4 (90%)	1.06×	0.003 (0%)	5.30×	23.7 (87%)	1.55×	51.6 (94%)	1.48×	3.10e+05			
		CH	1.75 (31%)	1.92×	21.7 (85%)	1.03×	0.003 (0%)	18.6×	29.7 (88%)	1.90×	64.1 (94%)	1.79×	1.77e+10			
		GL	1.42 (24%)	1.10×	12.0 (73%)	1.01×	0.003 (0%)	11562×	29.7 (87%)	1.09×	61.9 (93%)	1.08×	1.78e+08			
		CM	1.40 (24%)	1.36×	31.0 (87%)	1.06×	0.003 (0%)	1.63×	29.5 (87%)	1.35×	62.7 (94%)	1.32×	1.36e+05			
		OSM	1.45 (29%)	1.61×	25.2 (88%)	1.04×	0.003 (0%)	18.9×	29.5 (90%)	1.59×	63.0 (95%)	1.54×	6.48e+07			
		GeoMean	1.36 (27%)	1.51×	21.2 (85%)	1.08×	0.003 (0%)	30.0×	23.6 (86%)	1.50×	51.5 (93%)	1.45×	8.04e+06			
	High-dim.	MN	0.787 (15%)	1.68×	113 (96%)	1.02×	0.013 (0%)	1.73×	11.0 (73%)	1.68×	24.1 (86%)	1.66×	8.23e+05			
		BD	1.27 (22%)	1.77×	93.0 (96%)	1.02×	0.016 (0%)	1.88×	18.1 (82%)	1.77×	36.0 (90%)	1.72×	9.69e+06			
		RD	6.16 (39%)	1.79×	304 (97%)	1.02×	0.021 (0%)	1.83×	89.8 (91%)	1.79×	171 (95%)	1.76×	5.62e+04			
		ST	1.96 (22%)	1.48×	96.3 (93%)	1.05×	0.006 (0%)	1.47×	34.6 (84%)	1.48×	61.5 (90%)	1.43×	4.04e+10			
		IC	18.9 (57%)	1.66×	99.5 (90%)	1.06×	0.018 (0%)	1.64×	283 (96%)	1.66×	539 (97%)	1.54×	3.22e+08			
		OA	62.0 (32%)	1.78×	462 (78%)	1.03×	0.017 (0%)	1.77×	953 (88%)	1.78×	1969 (94%)	1.72×	9.85e+05			
		GeoMean	4.92 (29%)	1.69×	155 (91%)	1.03×	0.014 (0%)	1.72×	74.3 (85%)	1.69×	146 (92%)	1.63×	1.34e+07			

Table 6. Seeding-only running time and initialization SSE at k = 10000. The SSE columns report initialization SSE *relative to the best final SSE* (last column) achieved after refinement across all seeding methods; lower (closer to 1×) is better. Parenthesized percentages next to times are seeding's share of seed+refinement time; underlining marks the best value in each row. The time color scale excludes Random, whose near-zero seed time would otherwise dominate the baseline.

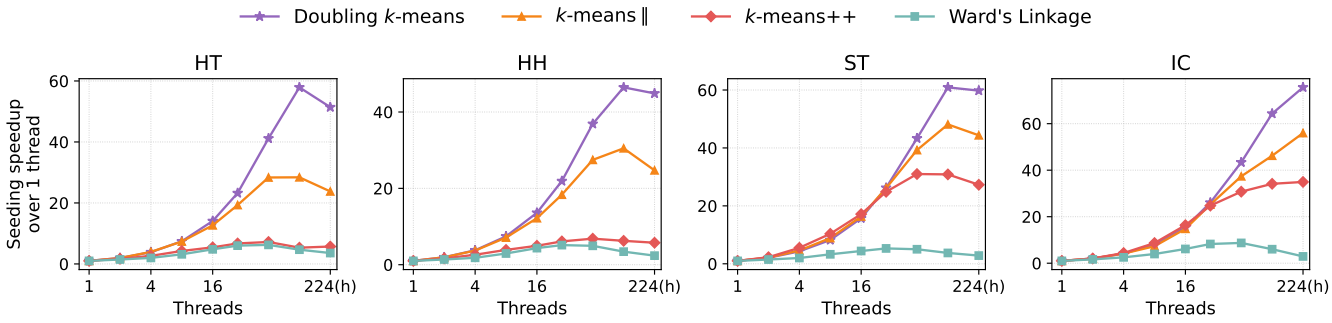


Figure 6. Seeding-only self-relative speedup versus thread count on four representative datasets at k=1000. Higher is better. (h) means hyperthreading is enabled.

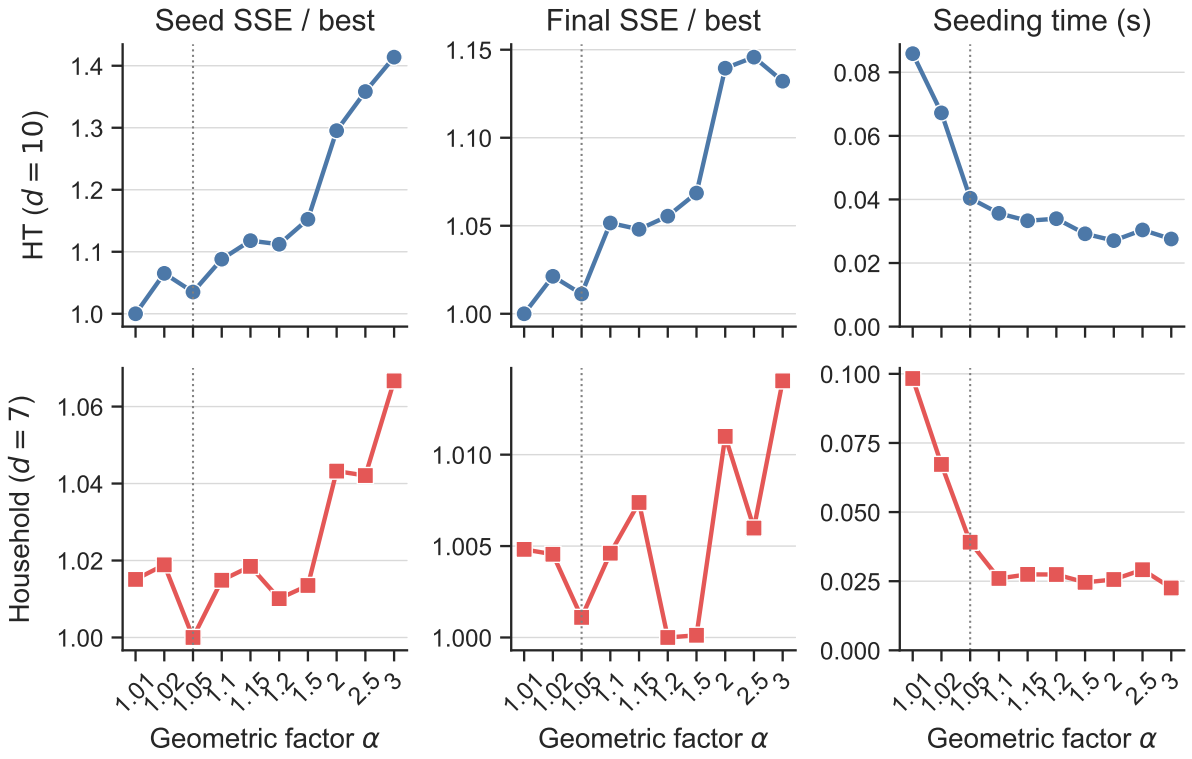


Figure 7. Seeding SSE, final SSE (each relative to its per-dataset best), and seeding time versus the geometric factor α in Doubling k-means. The dotted line marks the default α=1.05.

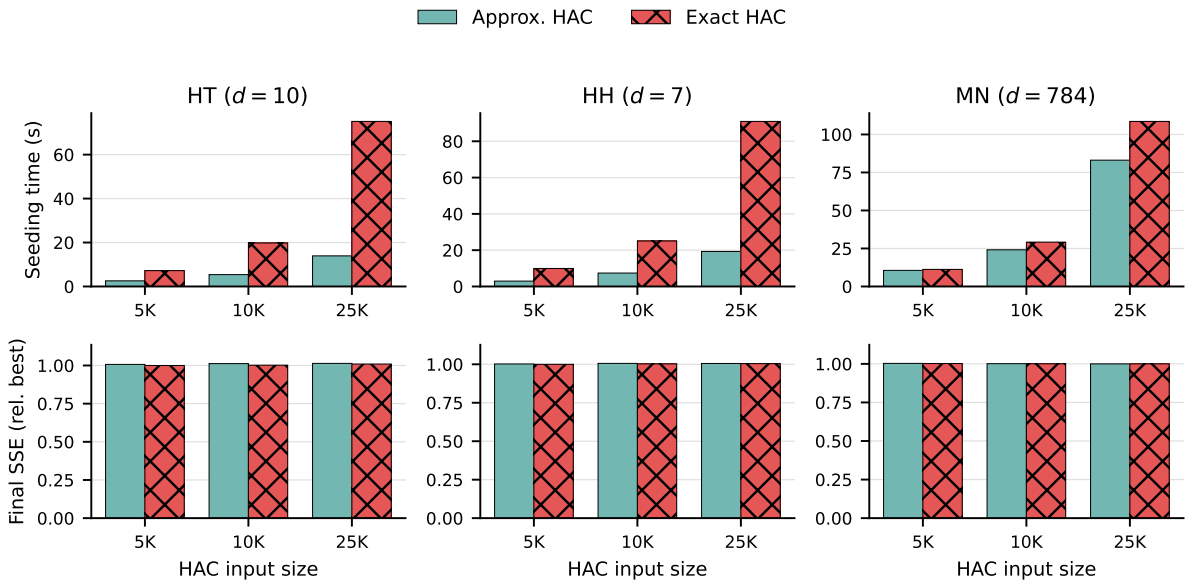


Figure 8. Approximate versus exact HAC across different HAC subsample sizes.

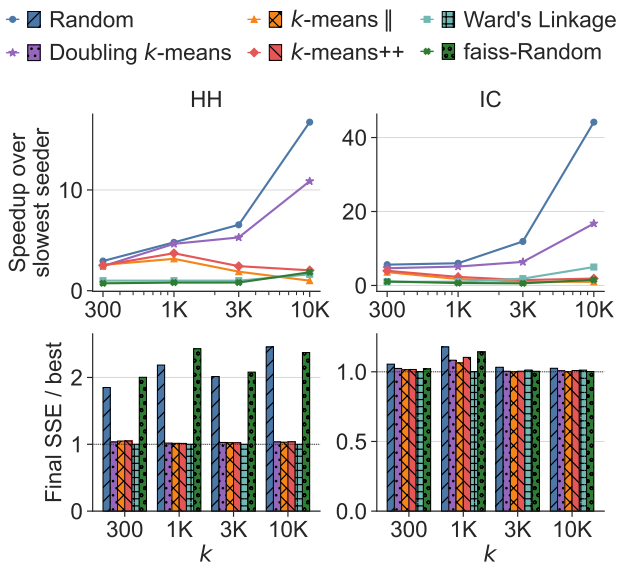


Figure 9. Speedup over the slowest seeder (top) and final SSE relative to the best seeder (bottom) versus k on HH and IC under ANNS refinement. Higher is better for speedup, lower for relative SSE.

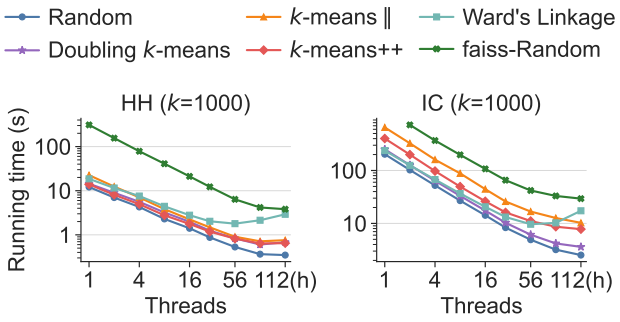


Figure 10. Total running time (seed + refinement) versus thread count on HH and IC at k=1000 under ANNS refinement. Both axes are log-scale; lower is better. (h) means hyperthreading is enabled. On IC, the faiss-Random curve starts at 2 threads because its single-thread run timed out.

Runtime-quality trajectories. Fig. 11 traces SSE against cumulative seeding and refinement time on HH and IC over the same 25 Lloyd iterations used in the main experiment. Diamonds mark the completion of seeding and therefore the start of refinement.

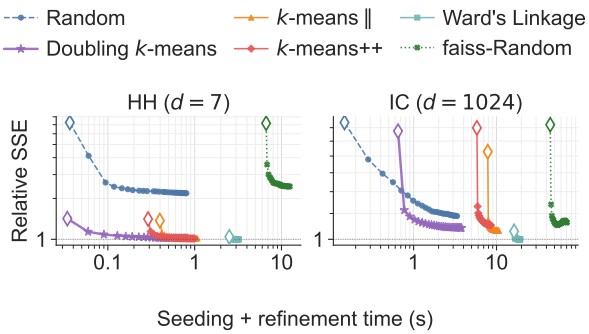


Figure 11. SSE versus cumulative seeding + refinement time over 25 **Lloyd's iterations on HH and IC at** k=1000**, relative to the best final SSE per panel.** Diamonds mark seeding completion. Lower and to the left is better.

Xiaojun Dong, Kishen N Gowda, Jessica Shi, MohammadHossein Bateni, Laxman Dhulipala, Xiaoen Ju, Jakub ÅĄÄĚcki„